

LOSTARK

기술 조사와 영상·기술서 구성안

화면 결과를 구현 원리와 연결하는 설명 체계

59개 기존 조사 항목 + 현재 코드의 핵심 경로 대조

영상 구성 · 자막 원칙 · 상세 기술서 · 포트폴리오 요약본

입장부터 전투까지, 그리고 그 장면을 만든 도구까지.

원리의 이름을 나열하는 대신 입력·처리·결과·검증을 설명합니다.

2026.10.02 / 1차 조사 원고

이 문서는 편집·집필 구성안입니다. 최종 제출용 포트폴리오가 아닙니다.

실제 영상 타임코드와 개인 기여는 별도 근거로 연결합니다.

읽는 순서

본문은 영상과 문서의 설명 체계를 정리하고, 부록은 기존 59개 기능 항목의 코드 근거를 다시 찾는 색인입니다. 먼저 02-03-05-16장을 읽으면 방향과 설명 깊이를 파악할 수 있습니다.

01. 이번 작업의 결론과 확인 범위	3
02. 선배 영상에서 가져올 형식과 현재 구현 사실	4
03. 개념을 어디까지 짚을 것인가	5
04. 프로젝트를 관통하는 세 가지 흐름	6
05. 주 영상의 권장 순서	7
06. 자막은 세 문장으로 설계한다	8
07. Effect Tool: 리본 하나를 깊게 설명하는 방법	9
08. Model-Anchor-Animation: 좌표계가 보이는 설명	10
09. Rendering: 기법 목록을 입력·출력으로 바꾼다	11
09A. PBR 설명의 실제 견본: 금속 표면이 달라 보이는 이유	12
10. 렌더링 영상의 정확한 비교 조건	13
11. Server-Party: 보이는 목록 뒤의 상태 소유권	14
12. 전투·이동·AI·제품 UI를 나누는 기준	15
13. 툴 설명 전체 목록과 촬영 우선순위	16
14. Save-Publish-Reload를 한 그림으로 설명한다	17
15. Profiler·메모리·로딩: 성과를 입증하는 장	18
16. 두 종류의 PDF는 역할이 다르다	19
17. 약 200페이지 기술서의 통합 구성	20
18. 바로 촬영·편집에 사용할 장면 후보	21
18A. 실제 영상 3개와 첫 번째 편집 지도	22
19. 오늘의 제작 순서와 완료 조건	23
20. 출처와 증거 원장의 사용법	24
영상 표본 / 로스트아크_최종1.mp4	25
영상 표본 / 로스트아크_최종영상1.mp4	26
영상 표본 / 로스트아크_최종영상2.mp4	27
부록 / 툴·파이프라인 기능 색인	28
부록 / Engine-Client 기능 색인	36
부록 / Server-Shared 기능 색인	41

01. 이번 작업의 결론과 확인 범위

이 프로젝트의 설명 단위는 **하나의 화면 결과를 만드는 입력·데이터·알고리즘·소유권·실패 처리**로 잡는다. 예를 들어 'Effect Tool' 한 장으로 끝내지 않고, 목록 선택 / 본 부착 / 리본 생성 / 재질 합성 / 저장과 다음 재생 반응을 각각 독립된 설명으로 나눈다. 반대로 벡터·행렬·JSON의 일반 이론을 장면마다 처음부터 반복하지 않는다.

영상은 입장 → 캐릭터 → 마을·파티 → 전투 → 그 장면을 만든 도구의 순서를 따른다. 자세한 개념과 수식은 대응 기술서로 이어지게 한다. 모든 도구는 전체 목록에 포함하되, 주 영상에서는 도구마다 같은 수준의 긴 설명을 반복하지 않고 실제 결과와 연결되는 작업을 보여준다.

참조한 이전 대화는 '포트폴리오와 엔진 수준 평가'다. 여러 프로젝트를 묶어 약 200페이지의 사례 연구형 기술서를 만들고, 영상·코드·측정 근거를 연결한다는 마지막 의도를 유지했다. 이전 Notion '홍건보 — Game Server / Client Programmer'의 프로젝트별 문제·설계·코드 근거 구조도 참고했다. 그 페이지는 7월 자료이므로 Winters의 수치·구조를 현재 LostArk 성과로 옮기지 않는다.

조사 기준은 현재 작업 폴더이며 HEAD는 `c44e8a6666ea9b4592ebf4873287444facc8db92`다. `git fetch` 후 HEAD는 origin/main보다 4개 커밋 뒤에 있으며, 작업 폴더에는 다른 작업의 미커밋 변경이 있다. 따라서 이 문서는 최신 main 배포본이나 현재 실행 EXE 전체를 평가한 자료가 아니다.

기존 59개 정적 조사 항목(도구·파이프라인 29, Engine·Client 19, Server·Shared 11)을 전범위 색인으로 재사용하고, 렌더링·이펙트·서버·파티·도구 진입점·게시 경로 등 대표 소비자를 현재 코드로 추가 대조했다. 전수 조사는 **기능 영역의 누락을 점검한 조사**이며, 모든 코드 줄·리소스·스킬 occurrence를 실행 검사했다는 뜻이 아니다.

완료: 이전 대화·Notion·선배 영상 기록 조사, 주요 코드 사실 교정, 개념 분해, 영상 순서, 두 종류 PDF의 설계, 장면 후보 목록. 사용자가 지정한 LostArkTransfer의 MP4 3개에서 메타데이터와 총 48개 표본 프레임을 확인했다. 미완료: 프레임 단위 인·아웃 타임코드, 최종 SRT, 편집본 export, 개인 기여 확정, 제출용 포트폴리오 완성. 이번 작업에서는 Client/UI 실행·촬영, 게임 빌드·게시, 렌더링 옵션 변경을 하지 않았다.

02. 선배 영상에서 가져올 형식과 현재 구현 사실

선배 영상의 장점은 플레이 중 보이는 현상을 설명하고, 마지막 도구 시연에서 그 현상을 만드는 과정을 보여주는 구성이다. 기술명과 구현 방식은 현재 코드로 다시 확인해야 한다. 저장소의 08-01 영상 분석 문서는 타 팀 영상에 대한 사용자 기록이며 우리 프로젝트의 구현 증거가 아니다.

자막 후보	현재 코드에 맞는 표현
Cascade 그림자를 구현했다	확인한 현재 경로는 단일 직교 투영 shadow map이다. CSM으로 소개하지 않는다.
모든 trail을 spline으로 만든다	특정 source ribbon의 curve 활성 분기에서 Hermite 보간을 사용한다. 선형 보간과 다른 trail 경로도 있다.
파티를 초대할 순서로 나열한다	서버가 수락을 처리해 가입시킨 순서로 roster를 만들고, 첫 멤버를 리더로 표시한다.
DataFiles의 이펙트 목록을 읽는다	Effect V1은 Data/Effects의 catalog와 authored 문서를 직접 소비한다.
PBR-SSAO-Bloom-FXAA는 모두 후처리다	PBR은 재질·조명 모델, shadow map은 그림자 기법, RT/MRT는 렌더링 중간 저장 구조다. 화면 공간 AO와 Bloom-FXAA는 별도 패스다.
선배처럼 IOCP-Protobuf를 쓴다	현재 서버의 근거는 Winsock TCP-select/nonblocking-session 송수신 thread와 Shared packet writer/reader다.
저장하면 모든 실행 상태가 바뀐다	Save·Publish·Reload/입장 승인은 별도다. 도메인마다 직접 소비와 게시 방식이 다르다.
새 최적화가 FPS를 개선했다	최신 변경은 compile/독립 검사와 제품 링크·동일 조건 FPS 검증을 분리해 기록한다.

추가로 실제 `Client_Defines.h`와 registry에는 `COLOSSEUM`이 있다. 일부 상위 문서의 enum 요약과 다르므로 콘텐츠 소개에는 현재 코드 목록을 사용한다. Profiler는 Debug의 F7로 창을 열고, 창 안에서 Capture를 제어한다. 모든 도구가 Debug 전용인 것도 아니다. 두 Boss Tool 진입점은 현재 Debug/Release 공통이다.

근거: Engine/Private/Shadow.cpp:119, Server/Private/GameRoom_PartyWorld.cpp:219, Client/Private/PartyWindowView.cpp:96, Client/Private/Effect_DocumentRenderer_Particles.cpp:1248, Client/Private/MainApp.cpp:11186-14605, Client/Public/Client_Defines.h:27.

03. 개념을 어디까지 쪼갤 것인가

한 설명 단위는 '하나의 원인과 관찰 가능한 결과'까지다. 독자가 이 장면을 보고 **무엇을 입력하면 어떤 규칙으로 무엇이 달라지는지** 말할 수 있으면 영상 설명을 마친다. 설계 선택의 이유·수식·예외 처리는 대응 기술서에서 계속한다.

깊이	설명할 내용	배치
화면 현상	어떤 결과를 보고 있는가	플레이 영상의 첫 자막
직접 원리	입력 → 핵심 처리 → 출력	다음 자막 또는 확대·도식
프로젝트 구현	실제 데이터·함수·소유자·저장 위치	도구 시연과 기술 소개서
설계 판단	대안, 선택 이유, 비용, 실패 시 동작	상세 기술서의 사례 본문
일반 이론	행렬, 보간, BRDF, TCP, 동시성	필요한 장에서 연결하는 공통 개념 부록

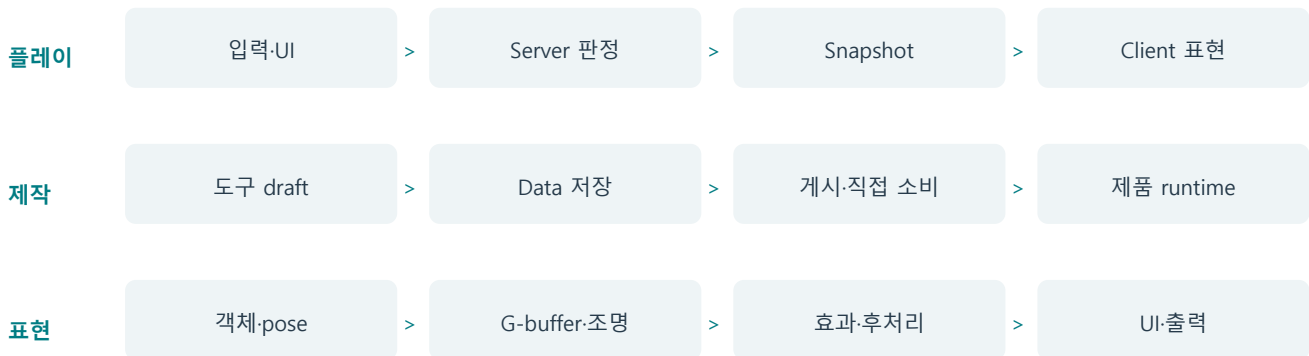
예를 들어 리본은 ① 이동 궤적의 점을 보존한다 ② 제어점 사이를 보간한다 ③ 폭·UV를 가진 띠 메시지를 만든다 ④ 재질과 blend로 합성한다 ⑤ 수명에 따라 제거한다고 나눈다. '리본 구현' 문장에 이 다섯 단계를 모두 몰아넣지 않는다.

본 부착은 ① 모델과 본 선택 ② 현재 pose의 본 행렬 취득 ③ 부착 offset과 owner world 합성 ④ local/world 재생 정책 ⑤ 잘못된 본·축·단위의 진단으로 나눈다. 벡터 내적이나 행렬 곱의 모든 정의는 별도 공통 개념 페이지에 둔다.

기법마다 원리를 늘어놓는 대신 장면 질문을 제목으로 쓴다. 'SSAO'보다 '벽과 바닥이 만나는 곳의 깊이감을 어떻게 보완했는가', '네트워크'보다 '네 명의 입장이 한 명의 실패로 갈라지지 않게 하는 방법'이 설명 범위를 정하기 쉽다.

수식은 결과를 예측하거나 버그를 설명할 때 넣는다. 수식을 제거해도 설명이 똑같다면 영상에는 필요하지 않다. 핵심 수식 하나, 그림 하나, 실제 코드 하나, 실패 사례 하나가 긴 함수 전문보다 유용한 경우가 많다.

04. 프로젝트를 관통하는 세 가지 흐름



플레이 흐름: 입력·제품 UI → typed command → Shared protocol → Server room의 검증과 시뮬레이션 → snapshot/event → Client replication → 모델·애니메이션·이펙트·HUD.

제작 흐름: 저작 도구의 draft → Data 정보 저장 → 도메인별 검증·게시 또는 직접 소비 → 실제 catalog/loader → 제품 재생. 저작 UI와 제품 UI의 소유권은 다르다.

한 프레임의 표현 흐름: 객체와 애니메이션 갱신 → 렌더 대상 제출 → 그림자·G-buffer·광원·투명 표현·후처리 → 제품 UI와 최종 출력. 정확한 패스 순서는 렌더링 장에서 현재 Renderer를 기준으로 설명한다.

이 세 흐름의 교차점을 대표 사례로 쓴다. 'Q를 눌렀다'는 입력·서버·presentation 사례이고, '그 Q의 effect를 수정했다'는 도구·데이터·재생 사례이며, '그 장면이 느렸다'는 CPU/GPU 계측 사례다. 같은 장면을 재사용하면 독자는 새 도구를 볼 때마다 게임 맥락을 다시 학습하지 않아도 된다.

Engine은 Prototype/Clone, Component, Level/Layer, CModel/CMaterial, renderer, profiler 등 범용 기반을 제공한다. Client는 LostArk의 화면·레벨·입력·presentation·저작 흐름을 소유한다. Shared는 packet과 gameplay 계약, Server는 월드 상태와 판정, Tools는 오프라인 검증·변환·게시를 맡는다.

현재 구조를 Winters의 ECS·JobSystem·RHI와 섞어 설명하지 않는다. LostArk의 대표 객체 모델은 Prototype/Clone·Component 구조다. 통합 기술서에서는 두 프로젝트가 요구 조건에 따라 어떤 선택을 했는지 비교한다.

05. 주 영상의 권장 순서

아래 시간은 편집 초안의 예산이다. 실제 촬영본의 타임코드가 아니다. 먼저 30분 안팎의 완결본을 설계하고, 길어지는 심화 시연은 챕터 영상으로 연결한다. 사용자가 원한 입장부터 전투, 마지막 도구 설명의 흐름을 유지한다.

순서·예산	보여줄 장면	기술 설명의 중심
도입 0:00~0:25	완성 플레이와 제작 도구를 짧게 교차	프로젝트·팀 역할·직접 담당 범위
입장 0:25~3:00	Lobby, 생성·선택, 실시간 Movie, Bern 진입	상태 수명, 서버 입장 승인, 실제 모델·카메라 재생
마을 3:00~6:00	이동, 파티 수락, HUD·인벤토리·NPC	서버 상태와 Client 표현, roster 순서
전투 6:00~12:00	대표 스킬, 발탄·쿠크 패턴, 지형 변화	명령·판정·연출의 역할, 타이밍·좌표
확장 12:00~14:00	Maharaka·Colosseum·Guide 중 대표 구간	같은 이동·전투 경로의 재사용, AI와 인간 구분
제작 14:00~26:00	Effect, Action, Animation, Map, UI, Camera	앞에서 본 장면을 실제로 수정하고 재생하는 과정
진단 26:00~29:00	Rendering Workbench, RT, Profiler	패스별 입력·출력과 측정의 해석
마무리 29:00~30:00	기술서 목차·역할·남은 과제	코드·문서·심화 영상으로 이동

전투 전체를 무자막으로 유지할 필요는 없다. 같은 규칙이 반복되는 구간은 줄이고, 패턴 시작·판정·반응처럼 인과관계를 볼 수 있는 앞뒤는 남긴다. 월드 전환 성공이나 실패 보존을 입증할 구간은 핵심 동작을 한 번 연속으로 보여준다.

모든 툴을 넣는 방법은 전체 목록 20~30초, 핵심 5~6개 작업의 충분한 시연, 나머지 도구의 짧은 결과 연결, 개별 심화 챕터로 구성한다. 전체를 길게 보존하고 싶다면 40~60분 확장본도 가능하나, 장별로 바로 이동할 수 있도록 챕터를 제공한다. 길이는 채용 규정이 아닌 편집 제안이다.

06. 자막은 세 문장으로 설계한다

각 장면의 자막 원고는 **관찰** → **구현** → **설계 이유** 세 문장을 먼저 작성한다. 한 화면에는 한 문장, 대체로 1~2줄을 시작점으로 삼는다. 한글 글자 수만으로 고정하지 말고 실제 재생 속도와 화면의 복잡도를 보고 체류 시간을 조절한다.

파티 장면 예시: '파티 수락 결과를 서버가 처리합니다.' → '서버가 확정된 가입 순서로 파티 목록을 갱신합니다.' → '클라이언트는 같은 roster를 표시하고 첫 멤버를 리더로 표시합니다.' 초대 순서와 가입 처리 순서가 다를 수 있음을 보여주는 비교는 심화 영상에 둔다.

리본 장면 예시: '이동 궤적을 따라 리본의 제어점을 기록합니다.' → '곡선 설정이 활성화된 구간은 Hermite 보간으로 세분화합니다.' → '길이에 맞춘 UV와 폭을 적용해 띠 형태의 메시를 만듭니다.' 실제 찍은 carrier가 그 분기를 사용하는지 asset-occurrence를 확인한 뒤 확정한다.

Effect V1 저장 장면 예시: '카탈로그의 stable ID로 이펙트 문서를 선택합니다.' → '저장 전 외부 변경을 검사하고 새 문서를 검증합니다.' → '이미 재생 중인 문서는 유지하고 다음 생성부터 저장된 표현을 사용합니다.' Save 버튼을 누른 화면만으로 마지막 결과를 입증하지 않는다.

자막에는 긴 함수명 대신 데이터의 역할을 쓰고, 확대 화면이나 기술서에 실제 심볼을 병기한다. '서버 동기화 구현' 대신 무엇을 보내고 누가 확정하는지 쓴다. '최적화 완료' 대신 줄인 처리와 측정 조건을 쓴다.

컷마다 장면 ID를 붙인다. 예: FX-02, NET-03, RENDER-04. 편집 원장에는 원본 파일, source in/out, timeline in/out, 자막, 기술서 절, 코드 근거, 검증 상태를 함께 둔다. 타임코드는 최종 편집에 맞춰 SRT/VTT로 만든다. 현재 원본의 구간 표본까지 확인했으며 최종 편집 시간이 없으므로 이 문서의 자막은 타이밍 없는 초안이다.

07. Effect Tool: 리본 하나를 깊게 설명하는 방법

실제 구현의 중심

Effect_DocumentRenderer_Particles.cpp에는 ribbon의 거리·접선 변화에 따라 subdivision 수를 정하고, curve 활성 시 XMVectorHermite, 그 외에는 XMVectorLerp를 사용하는 분기가 있다. 마지막 세분화점은 다음 제어점과 정확히 같게 유지한다. width·age·color·누적 거리도 연결한다.

Hermite는 두 위치 P0·P1과 두 접선 T0·T1, 구간 비율 t로 중간 위치를 결정한다. 기술서에는 '끝 위치를 유지하면서 접선으로 굽는 방향을 조절한다'는 그림을 먼저 두고 필요한 경우 다음 식을 넣는다.

$$P(t)=(2t^3-3t^2+1)P0+(t^3-2t^2+t)T0+(-2t^3+3t^2)P1+(t^3-t^2)T1$$

화면과 문서의 역할

영상: 같은 움직임에서 제어점, 곡선 보간, 완성된 리본을 순서대로 확대한다. 비교 촬영은 사용자가 승인한 테스트 입력에서만 수행한다. 실제 정상 프로젝트 설정을 임의로 바꾸지 않는다.

실질 기술 소개서: 제어점 저장 구조, 접선 계산, segment 길이·굽힘에 따른 subdivision 상한, 양옆 정점과 index, 누적 거리 UV, 수명 제거, 카메라 방향과 퇴화 입력을 설명한다. 그중 조사로 확인한 코드와 추가로 확인할 데이터 사례를 구분한다.

포트폴리오: 각진 궤적이나 과도한 세분화 같은 구체적 문제가 실제 자료에 있으면 문제·선택·결과 한 사례로 편집한다. 촬영 또는 로그가 없는 개선 전 모습을 만들어내지 않는다.

현재 주석은 원본 tangent 설정을 받아 프로젝트에서 보간한 구현이라고 설명한다. 원작 UE CPU 코드를 그대로 복원했다거나 모든 trail이 같은 spline을 사용한다고 쓰지 않는다. native ribbon program/carrier와 source profile 조건, Beam 제외, tangent flag, 거리 세분화 활성 조건을 함께 확인한다. baked EdgePairs는 이 중심선 보간을 사용하지 않는다. 접선은 인접 chord의 정규화 방향과 길이를 이용해 제한하며, 너무 긴 구간에서는 chord를 유지한다.

근거: Client/Private/Effect_DocumentRenderer_Particles.cpp:945-1218-1248. 개념 참고: Microsoft DirectXMath XMVectorHermite 공식 문서.

08. Model·Anchor·Animation: 좌표계가 보이는 설명

모델을 로드했다는 사실만으로 본 부착을 설명할 수는 없다. anchor가 객체 기준인지, 실제 skeleton의 특정 bone인지, 현재 pose를 따라가는지, spawn 시점의 위치를 고정하는지부터 구분한다.

화면에서는 실제 본을 선택하고 gizmo를 표시한 뒤 애니메이션을 재생한다. 이어 offset을 조정했을 때 모델의 움직임과 effect의 상대 위치가 어떻게 달라지는지 보여준다. 원작 알림의 scale과 사용자 조정 scale은 서로 다른 입력으로 설명한다.

기술서는 모델·본·socket·world 좌표계의 관계를 다룬다. 확인한 Tool consumer의 실제 합성은 $\text{SocketLocal} = S_{\text{socket}} \times R_{\text{socket}} \times T_{\text{socket}}$, $\text{AnchorWorld} = \text{SocketLocal} \times \text{BoneAnchorWorld}$ 다. source import-scale 정규화 분기는 RawBone의 3×3 basis만 정규화한 뒤 OwnerWorld를 곱하며, translation을 같은 배율로 다시 줄이지 않는다. 이 분기를 모든 모델의 일반 공식으로 확대하지 않는다.

필수 개념은 S/R/T, parent-child transform, bone pose, bind pose와 현재 pose의 차이, socket offset, local/world space, cm→m와 preScale이다. quaternion의 전체 유도나 모든 skinning 변형 기법은 부록으로 보낸다.

재료가 되는 구현은 CModel/CBone/CAnimation/CChannel과 CAnimationTargetService다. GPU skinning은 최대 4 bone influence의 weighted matrix와 offset × combined pose palette를 사용하는 경로가 있다. 최신 Movie sample reuse는 동일 channel 입력·시각의 local interpolation 결과를 재사용하며, clone의 clock-blend-root motion-combined matrix-skin palette는 독립 처리한다. 코드·독립 검사와 실제 GPU 통합·프레임 개선의 상태가 다르므로 완료 성과로 앞당기지 않는다.

실패 사례는 이름이 같은 본의 basis 차이, preScale 중복, 부모 scale이 particle 크기에 두 번 적용되는 경우, local/world 회전을 중복 적용한 경우다. 실제 기록이 있는 사례 하나를 골라 정상/오류 좌표축·수치를 비교하면 개념 설명이 구체화된다.

근거: Client/Private/Effect_Tool_Helpers.cpp:1327, Client/Private/Effect_PresentationService.cpp:4287, Client/Bin/ShaderFiles/Shader_VtxAnimMesh.hlsl:32, Engine/Private/Mesh.cpp:334, .md/GB/렌더링이펙트복원V2.md, 10-02 Movie animation sample reuse PLAN/RESULT.

09. Rendering: 기법 목록을 입력·출력으로 바꾼다

렌더링 장은 '이 화면을 만들기 위해 어떤 중간 이미지를 만들었는가'에서 시작한다. RT는 중간 결과를 저장하는 GPU 자원이고, MRT는 한 draw에서 여러 출력을 기록하는 방식이다. Deferred shading의 G-buffer를 설명할 때 색·법선·깊이·재질 입력의 실제 용도를 연결한다. Direct3D의 deferred context와 deferred shading은 다른 개념이다.

범주	기술서에서 설명할 것	추천 화면
장면 정보	G-buffer의 각 출력과 위치 복원	같은 카메라의 diffuse-normal-depth와 최종 화면
조명·재질	직접광, roughness/specular 계열, source material 분기	캐릭터/무기 재질과 입력 채널 확대
그림자	광원 공간 깊이, bias, 정적 캐시와 동적 갱신	shadow target와 지면 그림자
가시성·제출	frustum, LOD, static batch/instancing	같은 경로의 제출량·가시 객체 수
화면 공간 처리	SSAO, Bloom, tone mapping/LUT, FXAA	실제 활성 profile과 대상 pass
투명·특수 표현	blend-decal-왜곡-scene color 소비	동일 effect의 입력과 합성 결과

현재 source-family PBR와 RNM/IBL 등의 입력 경로가 있으나 모든 재질을 하나의 metal/roughness 공식으로 묶지 않는다. 현재 동적 그림자는 단일 직교 투영 방향광 shadow map과 3×3 PCF를 사용한다. 정적 caster는 행렬·객체·revision 조건이 유지될 때 캐시하고 동적 caster를 추가로 그린다. CSM은 별도 학습·향후 비교 항목으로만 둔다.

조사에서 확인한 SSAO는 half resolution, 12개 샘플, 5×5 depth/normal bilateral blur 경로다. Bloom은 표면·이펙트가 계산한 밝은 기여와 강도를 별도 버퍼에 모아 half-resolution 입력을 가로·세로 9-tap blur한다. BloomExtract의 입력에는 이미 contributor별 bright-pass가 반영돼 같은 threshold를 다시 적용하지 않는다. FXAA는 화면 색 경계의 계단 현상 완화를 다룬다. 구현의 존재를 현재 장면에서의 활성 상태로 바꾸어 말하지 않는다.

현재 주요 순서는 Shadow → NonBlend G-buffer → SSAO(선택) → Lights → SceneHDR 합성 → ScreenPosts → Bloom(선택) → Final의 tone mapping·grading·FXAA → DisplayOverlays → 일반 UI다. Portrait와 opaque picking depth는 별도 중간 처리다. SceneHDR 안의 선택적 SCENE_UI는 후처리에 참여하지만 일반 HUD는 최종 합성 뒤에 그린다.

근거: Engine/Private/Renderer.cpp, Engine/Private/Target_Manager.cpp, Engine/Private/Shadow.cpp, Engine/Bin/ShaderFiles/Shader_Deferred.hlsl. 일반 개념: Microsoft Output-Merger/CSM, NVIDIA FXAA whitepaper.

09A. PBR 설명의 실제 견본: 금속 표면이 달라 보이는 이유

이 절은 전체 재질의 공통 공식을 선언하는 것이 아니라, 확인한 map PBR family 하나를 설명하는 견본이다. 화면의 금속 표면을 먼저 보여준 뒤 입력 채널 → 재질 값 → 직접광 → 환경 입력 → HDR 결과의 순서로 분해한다.

`Shader_MapMaterialSurface.hlsl:495`에서는 ORM texture의 B를 metallic, G를 roughness, R을 AO로 읽고, 각각 intensity·power와 clamp 등의 처리를 한다. 색 이미지만 있어서는 같은 재질이 나오지 않는 이유가 이 입력의 차이에 있다.

같은 경로의 specular 입력은 $F0 = \text{lerp}(0.08 \times \text{saturate}(\text{specularIntensity}), \text{albedo}, \text{metallic})$ 형태다. 영상에서는 '금속도에 따라 반사색의 기준을 비금속 값과 표면색 사이에서 바꿉니다'라고 설명하고, 기술서에는 각 채널·범위·실제 material descriptor를 붙인다. 모든 캐릭터가 이 식을 쓰는 것은 아니다.

`Shader_Deferred.hlsl:366`의 직접광은 normal·view·light·half vector와 roughness 기반 분포, Fresnel, visibility를 사용하며 source의 geometric-normal 보정과 specular peak cap도 갖고 있다. 따라서 일반 Cook-Torrance 예제의 한 식만 복사해 현재 shader 전체를 설명하면 세부 차이가 사라진다.

간접 입력도 구분한다. RNM은 회수한 방향성 lightmap을 소비하는 경로이고, 환경 반사는 BRDF lookup과 roughness에 따른 cube mip 조회를 사용한다. 이 코드가 있다는 사실을 자체 Lightmass 구현이나 원작 간접광 전체 복원으로 설명하지 않는다.

영상 자막 초안: '이 재질은 색 외에도 ORM의 세 채널을 사용합니다.' → '금속도와 거칠기가 반사색과 하이라이트의 모양을 바꿉니다.' → '구운 조명과 환경 반사 입력을 함께 합성합니다.' 촬영본의 실제 material family를 확인한 뒤에만 이 자막을 붙인다.

이 한 사례를 상세 기술서에서는 2~4페이지로 확장한다. 1쪽은 화면과 채널, 2쪽은 데이터와 수식, 3쪽은 shader와 RT 소비, 4쪽은 발견한 오류·수정·검증이다. 오류 자료가 없으면 가짜 전후 비교를 만들지 않고 현재 동작 설명으로 마친다.

근거: Client/Bin/ShaderFiles/Shader_MapMaterialSurface.hlsl:495-510-643-666-681, Engine/Bin/ShaderFiles/Shader_Deferred.hlsl:366-422.

10. 렌더링 영상의 정확한 비교 조건

촬영된 화면에 Bloom이 예뻐 보인다는 이유로 해당 pass가 활성이라고 단정하지 않는다. 재질의 emissive와 blend만으로 유사한 인상이 생길 수 있다. 현재 scene, quality override, region, 저장 profile과 실행 상태를 함께 확인한다.

쿠크의 현재 base profile에는 SSAO-Bloom-FXAA가 꺼진 값이 있다. 다른 profile이나 전역 스위치가 켜져 있어도 실제 scene override와 최종 shader 분기에 따라 적용이 달라진다. FXAA에는 subpixel 값이 0일 때 우회하는 경로도 있다. 촬영을 위해 팀장 저장값을 임의로 켜거나 덮어쓰지 않는다.

A/B 촬영은 사용자가 선택한 비교용 설정에서 같은 해상도·카메라·시간·노출·광원·움직임을 유지하고 한 항목씩 비교한다. 실제 영상의 FXAA OFF 장면은 '이 장면에 FXAA가 적용됨'의 근거로 사용할 수 없다. 기능이 켜진 검증 장면이 없으면 기술서의 구현 근거로 소개하고 화면 적용을 주장하지 않는다.

CSM은 카메라 시야를 구간으로 나누고 각 구간의 shadow map을 사용하는 기술이다. 현재 코드의 단일 light view/projection과 구분해야 한다. 'Cascade'라는 이름이 원본 UE3 particle 자료에 있다고 CSM 구현으로 연결하지 않는다.

각 rendering 비교에는 '원리 설명용 입력', '실제 제품 기본값', '측정 시 입력'을 따로 표기한다. 화면 품질 비교와 GPU 시간 비교도 따로 기록한다. 현재 셰이더의 존재는 원작과 시각적으로 동일하다는 증명이 아니다.

근거: Data/Rendering/Authoring/RenderingProfiles.json, Engine/Bin/ShaderFiles/Shader_Deferred.hls:1679, .md/GB/10-02/2026-10-02_SAVE_PUBLISH_RUNTIME_GUIDE.md. CSM 정의: <https://learn.microsoft.com/en-us/windows/win32/dxtecharts/cascaded-shadow-maps>

11. Server-Party: 보이는 목록 뒤의 상태 소유권

일반 파티의 초대자는 파티가 처음 만들어질 때 첫 멤버로 들어간다. 수락을 처리할 때 responder를 vector 뒤에 추가하고, 그 순서를 roster로 보낸다. Client는 수신 순서를 표시하며 첫 멤버를 leader로 표시한다. 따라서 '초대 발송 순서'와 '서버의 가입 확정 순서'는 다를 수 있다.

영상에서는 두 Client의 같은 파티 목록과 체력 갱신을 연결한다. 심화 장면에서는 A·B를 차례로 초대한 뒤 수락 순서를 바꾸어 실제 가입 처리 결과를 비교할 수 있다. 이는 촬영 계획이며 이번 조사에서 재현한 결과는 아니다.

서버는 30Hz 고정 간격의 room loop를 사용한다. 입력 command가 room의 검증과 상태 변경을 거쳐 snapshot으로 표현된다. Client는 표현 보간과 제한된 local 이동 예측을 갖고 있으므로 '아무 예측 없이 서버 응답만 기다린다'고 쓰지 않는다.

TCP는 메시지 단위가 보장되는 통로가 아니므로 packet frame/parser가 stream에서 packet 경계를 복원한다. 전송 큐는 reliable 메시지의 순서와 snapshot의 최신성을 구분한다. 아직 송신되지 않은 오래된 snapshot을 최신 것으로 병합하는 것이 TCP 자체가 패킷을 임의로 버린다는 뜻은 아니다.

현재 transport는 선배 영상의 IOCP-Protobuf 설명과 구분한다. 여러 thread가 있다는 사실만으로 lock-free나 대규모 동접 처리를 구현했다고 소개하지 않는다. AI 20명도 실제 network client 20명 부하 검증과 다르다.

대표 심화 사례는 파티 전원 이동이다. 모든 대상의 입장과 초기 송신 준비가 끝나기 전에 source membership을 바꾸지 않는 경로를 설명한다. 성공 화면과 한 명의 admission 실패에서 기존 방·파티가 유지되는 화면을 함께 확보하면 설계 의도가 보인다.

근거: Server/Private/GameRoom_PartyWorld.cpp:219-226-242-1069, Server/Private/ServerApp.cpp:2708, Server/Private/ClientSession.cpp:217-533-568, Shared/Public/Network/PacketStreamParser.h, Client/Private/Character.cpp:2078-2127.

12. 전투·이동·AI·제품 UI를 나누는 기준

전투: 물리 키 → class와 inputSlot으로 skillId resolve → Server의 cooldown-resource-action 검증 → combo/action 확정 → Client animation/effect/HUD. 한 스킬을 이 흐름으로 끝까지 설명한 뒤 다른 직업의 차이를 보여준다. 모든 스킬의 동일한 packet 설명을 반복하지 않는다.

레이드: gameplay의 판정 시간과 presentation의 animation/effect/camera 시간을 구분한다. 보스의 패턴·월드 파괴·combat object·무력화·관문 전환을 stable ID와 server tick으로 연결하는 사례를 고른다. 화려한 장면만으로 데미지 권위나 동기화 성공을 주장하지 않는다.

이동: 현재 Server navigation은 grid 기반 8방향 A*, corner-cut 방지, 높이·traversal 확인과 경로 보정이 핵심이다. navmesh/Detour로 소개하지 않는다. 경로 찾기 성공, 최종 목표가 walkable인 것, 실제로 연결되어 도달 가능한 것은 서로 다르다.

AI: Guide, Waterpang, Colosseum 용병은 기능과 수명이 다르다. 기존 player 이동·스킬·피격·복제 경로를 재사용한다는 점이 설명거리다. 가짜 인간 session을 만들거나 생성형 모델이 판단한다고 표현하지 않는다. 행동 규칙과 설정 변화가 화면에서 어떻게 드러나는지 보여준다.

제품 UI: authoring의 ImGui와 CUIObject 계열 제품 image widget을 구분한다. reference resolution의 rect-stable slot-draw order-Resources 상대 asset ID로 배치하며, 화면 좌표 picking과 입력 소비 후 gameplay command 차단을 설명한다. UI가 직접 packet을 보내거나 피해를 계산하는 구조로 묘사하지 않는다.

캐릭터·경제: roster와 복원 상태는 process-session 메모리다. 재실행 후 영구 계정 저장이나 DB 경제 시스템으로 확대하지 않는다. 인벤토리·장비·재화 변경의 검증 권위는 Server 경로로 설명한다.

근거: Server/Private/PlayerSkillSystem.cpp, Server/Private/ServerNavigation.cpp, Server/Private/GameRoom_Inventory.cpp, Client/Private/PlayerController.cpp, Shared/Public/Network/PacketType.h, .md/TEAM/TEAM_GAMEPLAY_INTERFACE_HANDBOOK.md.

13. 툴 설명 전체 목록과 촬영 우선순위

도구 전체를 보여주되, UI 창의 개수를 기술 성과로 삼지 않는다. 각 도구가 소유하는 입력, 저장하는 데이터, 소비하는 runtime을 연결한다. 아래 목록과 부록의 29개 항목이 도구·오프라인 pipeline 범위를 이룬다.

류음	포함할 실제 도구·기능	영상에서 증명할 작업
이펙트 제작	Effect V1, Effect V2	선택·편집·본 부착·재생·저장·다음 생성
행동 제작	Animation Clip, Action Workbench, Sequencer	clip/skill 연결과 effect-collider-sound-camera-light 시간 배치
보스 제작	Valtan/Kouku Boss, Valtan Logic Pattern, Balance	pattern/flow·수치와 Server 승인 재생
월드 제작	Map, World Level, Object, Navigation·Destruction	배치·stable ID·구역·서버용 이동 데이터와 연출
화면 제작	Camera, HUD Layout, Equipment Authoring	카메라 key, UI 배치, 외형 preview
AI 제작	DimensionMaster Guide, Waterpang AI	데이터 편집·revision·Apply/Save 권위
진단	Rendering Workbench, Composition Profiler	RT·재질 기여와 CPU/GPU 구간
오프라인	extract/cook, domain publishers, build/package, validators, UI generator	입력→검증→출력과 실제 consumer

별도 Sound Tool이 있다고 나열하지 않는다. 현재 런처 기준 사운드는 Action/Effect 시퀀서의 lane과 연결 writer-runtime 으로 설명한다. 독립 Sequence의 local preview/Save와 실제 제품 연결도 구분한다. 29개는 독립 GUI 창의 개수가 아니라 도구·오프라인 pipeline 조사 항목 수다.

우선 긴 시연을 배정할 후보는 Effect, Action/Animation, Map, Rendering, Profiler다. 나머지도 전체본에 포함하고, 앞선 게임 장면과 겹치는 원리는 링크로 연결한다. 툴 하나당 '대상 선택 → 한 값 편집 → 저장 경계 → 실제 재생 → 실패 또는 취소'의 작업을 기본 단위로 삼는다.

근거: Client/Private/MainApp.cpp:14605, Client/Public/CompositionWorkbenchSession.h, .md/TEAM/ANIMATION_TOOL_OWNER_HANDOFF.md, 기존 Portfolio-Tools-Inventory.json.

14. Save·Publish·Reload를 한 그림으로 설명한다

Save는 메모리 draft를 저작 정보에 보존하고, Publish는 제품 계약에 맞게 검증·변환한 파일을 만든다. Reload 또는 admission은 실행 중 메모리가 새 상태를 쓰게 하는 별도 단계다. Build는 C++·HLSL에서 EXE·DLL·CSO를 만드는 작업이다.

사례	정보와 소비 경로	영상·문서의 핵심
Effect V1	Data/Effects catalog + Authored 직접 소비	저장 이후 신규 spawn과 기존 occurrence 수명 구분
Map	Imported catalog + Authoring → DataFiles/Map	asset 정의와 placement 인스턴스, scope와 rollback
Rendering	Authored profile → runtime JSON → service Reload	직렬화와 게시, 실행 적용의 차이
Gameplay	Balance 등 → Gameplay.bootstrap → Server catalog	schema-revision·참조와 staged 교체
Valtan	source → candidate / 파일 게시 / live admission	마지막 정상 제품과 편집 중 source의 공존
Waterpang	typed 요청 → Server revision 검증 → Apply/Save	파일 소유자가 Server인 예외

도구의 데이터 목록은 catalog 정의이고 실제 화면의 여러 객체는 placement 또는 occurrence다. 같은 asset을 여러 곳에 쓰더라도 저장 ID를 포인터나 vector index로 삼지 않는 이유를 여기서 설명한다.

실패 처리는 ‘atomic’이라는 단어 하나로 끝내지 않는다. 단일 파일 교체, 여러 파일의 실패 시 역순 복구, process crash까지 보장하는 transaction은 서로 다르다. domain별 코드가 보장하는 수준까지만 소개한다. 모든 UI reader가 동일한 strict 실패 정책을 지키는 것도 아니다.

대표 시연은 정상 저장 한 번, 잘못된 입력 또는 외부 변경 거절 한 번, 기존 상태 보존 확인 한 번이다. 실제 편집 중인 사용자의 draft를 버리는 reload로 시연을 만들지 않는다.

근거: .md/GB/10-02/2026-10-02_SAVE_PUBLISH_RUNTIME_GUIDE.md, Client/Private/Effect_Tool_DocumentIo.cpp:302-355-432, Tools/MapPipeline/Publish-MapAuthoring.ps1, Client/Private/RenderingProfileService.cpp.

15. Profiler·메모리·로딩: 성과를 입증하는 장

Profiler 소개는 'FPS를 띄웠다'보다 CPU 구간의 inclusive/self, GPU timestamp, frame interval, 기다림과 유효 표본을 어떻게 구분하는지에 초점을 둔다. 누락된 CPU 자식 scope가 있으면 Self를 확정할 수 없고, GPU pending을 0ms로 평균해서도 안 된다.

현재 기록에는 빠른 기본 120프레임 평균 interval 76.127ms, 빠른 무비 126.165ms 등이 있다. 이는 기존 Debug capture 분석값이며 이번 작업의 새 측정이나 Release 성능이 아니다. 무비에는 raw CPU 누락도 있어 근거의 범위를 제한한다. 서로 다른 scene과 Detailed 설정의 결과를 전후 개선처럼 나란히 놓지 않는다.

최신 source에는 계측 신뢰도, visibility 평가, animation sample reuse 관련 변경이 있으나 제품 링크·설치·동일 장면 FPS가 남은 항목이 있다. 따라서 지금은 '병목 조사와 개선 후보 검증' 사례이고, 완료된 성능 개선율을 약속하는 사례는 아니다.

메모리 장은 Prototype/Clone 소유권, shared/weak reference, 공유 texture cache, level 해제, 로더 취소와 bounded join을 설명한다. Profiler의 texture memory counter는 생산자 부재로 N/A인 경계가 있으므로 완성된 VRAM 추적 도구로 소개하지 않는다.

좋은 성능 사례에는 hardware, 빌드 구성, 해상도·quality, 카메라 구간, warm-up, frame 범위, 표본 수, 계측 모드, 평균·p95·최대, 전후 코드 identity가 필요하다. 같은 조건으로 얻은 변화만 성과 수치로 쓴다. 로딩 준비 시간과 지속 frame 비용도 분리한다.

근거: .md/GB/10-02/2026-10-02_FRAME_PIPELINE_OPTIMIZATION_RESULT.md, .md/GB/10-02/2026-10-02_CLASS_MOVIE_PERFORMANCE_RESULT.md, Engine/Private/Profiler.cpp, Engine/Private/Material.cpp, Client/Private/ProfilerTool.cpp.

16. 두 종류의 PDF는 역할이 다르다

실질 기술 소개서는 다른 개발자와 본인이 구현을 따라갈 수 있는 문서다. 실제 파일·자료구조·단위·함수 책임·제어 흐름·수식·실패 처리·재현 방법을 담는다. 설명 단위마다 대표 코드와 입력 예제가 필요하며, 전체 코드 복사로 페이지를 채우지 않는다.

포트폴리오 기술 소개서는 독자가 짧은 시간에 개인의 판단과 기여를 평가하는 문서다. 기능의 목적, 직접 담당 범위, 핵심 구조도, 대표 문제 3개, 결과와 한계, 심화 링크를 담는다. 둘은 같은 근거 원장에서 만들되 문장과 편집 밀도는 다르게 한다.

포트폴리오 요약본 예시	페이지 예산
프로젝트·개인 역할·대표 영상	2
플레이 범위와 시스템 구조	2
이펙트·렌더링 대표 사례	5
서버 권위·파티 전환 대표 사례	4
저작→저장→실행 대표 사례	4
계측·성능 분석과 검증 한계	3
팀 협업·라이브러리·출처·심화 목차	2
합계	22

22페이지는 초안 예산이다. 근거가 없는 부분을 채우기 위해 분량을 고정하지 않는다. 대표 기여가 달라지면 사례 배분을 바꾼다.

개인 역할의 출발 기록은 08-01의 '건보: 프레임워크·이펙트·렌더링·서버'다. 이것만으로 현재 모든 기능을 직접 구현했다고 판정할 수는 없다. 각 사례에 직접 설계/구현, 공동 작업, 통합·검증, 외부 라이브러리, 원본 게임 자산의 역할을 따로 기록한다. AI 지원도 실제 작업 방식에 맞게 설명한다.

17. 약 200페이지 기술서의 통합 구성

이전 대화의 목표는 LostArk 한 프로젝트를 억지로 200페이지로 늘리는 것이 아니다. Unreal 기반 프로젝트, StarCraft, 팀 프로젝트, WintersEngine, LostArk에서 같은 질문에 어떻게 다른 답을 냈는지 묶는 사례 연구형 기술서다.

장	페이지 예산	중심 질문
전체 개요·역할·증거 읽는 법	8	무엇을 만들었고 무엇을 검증했는가
프로젝트 계보와 요구 조건	12	프로젝트가 바뀌며 어떤 제약이 달라졌는가
런타임·객체·메모리·로딩	22	누가 객체와 자원의 수명을 책임지는가
입력·서버·전투·월드 전환	28	상태의 최종 결정권과 실패 처리는 어디에 있는가
렌더링·재질·조명	30	데이터와 패스가 화면을 어떻게 만드는가
애니메이션·이펙트·좌표계	24	시간과 공간을 공유해 표현을 어떻게 연결하는가
도구·데이터·검증·게시	24	팀이 만든 콘텐츠를 어떻게 안전하게 실행하는가
Profiler·성능 사례	22	관찰과 가설을 어떻게 구분하고 검증했는가
대안·실패·한계·통합 논의	14	다른 선택의 비용과 남은 문제는 무엇인가
용어·코드/영상 인덱스·참고 자료	16	근거를 어떻게 다시 찾는가
합계	200	집필 예산이며 완성 분량의 약속은 아님

각 장의 반복 구조는 문제와 성공 조건 → 필요한 개념 → 실제 설계·흐름 → 핵심 구현 → 실행/측정 → 비용·한계다. 일반 이론의 공통 페이지를 링크해 중복을 줄인다. 같은 프로젝트의 클래스 목록을 앞에서부터 설명하는 방식은 피한다.

Winters의 ECS·JobSystem·리플레이·백엔드, Unreal 프로젝트의 완주 경험은 기존 Notion에서 후보를 가져올 수 있다. 그 근거를 최신으로 다시 검증하기 전에는 LostArk와 비교한 사실을 새로 확정하지 않는다. 이번 원고는 LostArk 조사와 통합 목차까지를 담당한다.

18. 바로 촬영·편집에 사용할 장면 후보

아래는 기술별 촬영·편집 후보다. 기존 영상에서 확인한 장면은 다음 18A절에 연결했고, source in/out과 timeline in/out은 해당 구간의 연속 재생과 편집 후 입력한다. 이미 찍힌 장면은 활용하고 부족한 증거 장면을 보완한다.

ID	장면·보여줄 증거	자막의 주장
ENTRY-01	캐릭터 선택과 Bern 진입의 연속 구간	생성·입장 성공 시점과 session 상태
MOVIE-01	카메라·배우·이펙트가 바뀌는 소개 장면	실시간 scene 구성과 시간 샘플링
NET-01	서로 다른 Client의 이동·스킬	Server 상태와 Client 보관·표현
NET-02	초대·수락·같은 roster-leader 표시	서버가 확정한 가입 순서
NET-03	파티 전원 이동 또는 admission 실패	준비 후 commit, 실패 시 기존 상태
COMBAT-01	스킬 입력·모션·이펙트·피해·HUD	command부터 presentation까지의 연결
RAID-01	패턴의 전조·판정·파괴·phase	gameplay와 연출의 같은 식별자·시간
FX-01	V1 목록·문서 선택·저장·다음 재생	catalog와 direct authored, occurrence 수명
FX-02	해당 ribbon의 제어점·곡선·최종 띠	조건부 Hermite 보관·subdivision
FX-03	실제 본·offset·움직이는 모델	pose와 owner basis를 사용하는 부착
TOOL-01	clip·effect·sound·camera 시간 편집	공통 sequencer와 domain별 문서
MAP-01	배치·저장·publish·제품에서 소비	asset ID와 placement ID의 분리
UI-01	HUD rect 편집과 제품 widget	authoring과 runtime UI의 분리
AI-01	AI 설정 조회·Apply·revision 응답	Server 소유 상태와 변경 검증
RENDER-01	RT 보기와 동일 장면 최종 합성	각 패스가 만드는 중간 정보
RENDER-02	승인된 동일 조건 A/B	실제 적용 상태를 확인한 기법 비교
PERF-01	Capture·느린 frame·scope·JSON	병목과 계측 한계의 근거

개별 클립의 구성은 원래 장면 3~5초 → 주목 지점 표시 → 개념/구조도 → 조작 또는 비교 → 결과 유지다. 길이는 실제 자막 읽기와 동작 속도에 맞춰 조절한다. 속도를 바꾼 장면에는 배속을 표시하고 네트워크 지연·성능 증명 구간은 원속도를 보존한다.

18A. 실제 영상 3개와 첫 번째 편집 지도

세 파일의 총 길이는 1시간 59분 54.550초다. 모두 3840×2160 H.264 영상과 48kHz AAC 음성 트랙을 갖고 있으며 영상 트랙은 60fps다. 이는 인코딩 형식이고 실제 게임이 60FPS로 실행됐다는 증거는 아니다. 각 영상에서 16개씩 균등 표본을 추출해 직접 열람했다. 전체 순차 재생이나 음성 전사는 하지 않았다.

파일·길이	확인한 원본 표본 시각	편집 역할
로스트아크_최종1.mp4 / 42:52.367	03:21 클래스 Movie, 06:02 외형 편집, 08:43 Bern, 11:23 설정·파티, 14:04·16:45·19:26·22:06 발탄, 27:28 이후 쿼크 표본	입장·캐릭터 소개의 앞부분과 레이드 기본 흐름
로스트아크_최종영상1.mp4 / 1:08:53.433	35:31 카드미로, 48:26 횡스크롤 관전, 52:45 빙고, 57:03 항해, 1:01:21 워터 팡	첫 파일과 겹치는 레이드는 비교 선택, 후반 확장 콘텐츠 보강
로스트아크_최종영상2.mp4 / 8:08.750	00:38 입장 수락, 01:09 VS 로딩, 02:10 도열, 02:40~04:12 전투 표본, 04:43 복귀, 05:44~07:46 용 탑승 표본	콜로세움의 입장·전투·복귀와 이동 연출 사례

위 시간은 원본에서 관찰한 지점이며 연속 구간의 정확한 시작·끝이나 최종 타임라인 위치가 아니다. 파일명이 '최종'이라는 이유로 자동 채택하지 않는다. 최종1과 최종영상1의 겹치는 레이드 장면은 camera·UI·실수·대기 구간을 비교해 선택한다.

48개 표본에서는 저작도구 창을 확인하지 못했다. 영상 전체에 도구가 없다고 판정한 것은 아니지만, 현재 확보한 근거는 제품 플레이·UI·연출에 집중되어 있다. Effect/Map/Rendering/Profiler의 편집·저장·재생 과정을 보여줄 촬영물을 더 찾거나 보완하는 것이 다음 핵심 작업이다.

최종영상1의 44:08 표본은 어두운 관전 화면이다. 사망·관전 기능 설명에 쓸 의도가 없다면 해당 구간을 우선 검토해 줄인다. 반대로 48:26 관전 중 기믹 화면은 관전 기능을 설명하는 별도 컷으로 활용할 수 있다.

파일 크기와 수정시각은 표본 추출 전후 동일했고 moov 파싱 및 분산 디코드는 성공했다. 전체 파일 무결성이나 export 프로그램의 완료 상태를 보증한 것은 아니다. 원본을 수정하지 않았다. PDF의 영상 표본 부록과 out/PortfolioAudit20261002/media/visual_observations.json에 관찰을 보존했다.

19. 오늘의 제작 순서와 완료 조건

첫째, export가 완료된 원본과 기존 촬영본을 그대로 보존하고 파일·길이·해상도·FPS·내용을 목록화한다. 영상에서 확인할 수 없는 알고리즘은 코드 근거와 연결한다. export 중인 파일을 완성본으로 읽거나 수정하지 않는다.

둘째, ENTRY→NET→COMBAT→TOOL→RENDER/PERF 순서로 무자막 러프컷을 만든다. 기존 촬영본에서 필요한 앞뒤를 남기고 반복·대기 구간을 줄인다. 최종 길이가 정해지기 전 SRT를 고정하지 않는다.

셋째, FX-02·NET-02·MAP-01 또는 TOOL-01의 세 사례부터 자막·도식·기술서 한 절을 함께 완성한다. 이 세 개가 전체의 설명 밀도와 시각적 형식을 정하는 샘플이 된다.

넷째, 같은 근거로 실질 기술서의 상세 전체본을 유지하며, 별도의 22페이지 포트폴리오 요약본을 만든다. 개인 기여를 확인한 항목만 1인칭 성과로 쓴다. 나머지 도구와 게임 콘텐츠는 전체 기능 지도와 심화 챕터에 연결한다.

다섯째, 최종 영상 타임라인에서 SRT/VTT를 생성하고 한글 줄바꿈·겹침·읽기 시간을 재생 검토한다. PDF는 목차·코드/영상 링크·이미지 해상도·글꼴·페이지 넘침을 확인한다. 자막의 주장과 해당 장면·코드가 맞는지 마지막으로 대조한다.

이번 조사 PDF의 종료 조건은 코드 근거·설명 깊이·편집 순서·두 PDF 목차가 서로 연결되어 있는 것이다. 최종 제출물의 종료 조건은 실제 영상 파일과 타임코드, 사용자 기여 확정, 검증된 시연과 수치가 붙는 것이다. 둘을 구분해 진행한다.

20. 출처와 증거 원장의 사용법

코드 경로는 `C:/Users/tnest/Desktop/LostArk` 기준이다. 줄 번호는 조사 시점의 현재 작업 폴더 기준이며 미커밋 변경이 포함된다. 외부 제출 시에는 최종 승인된 commit의 코드 링크로 고정한다.

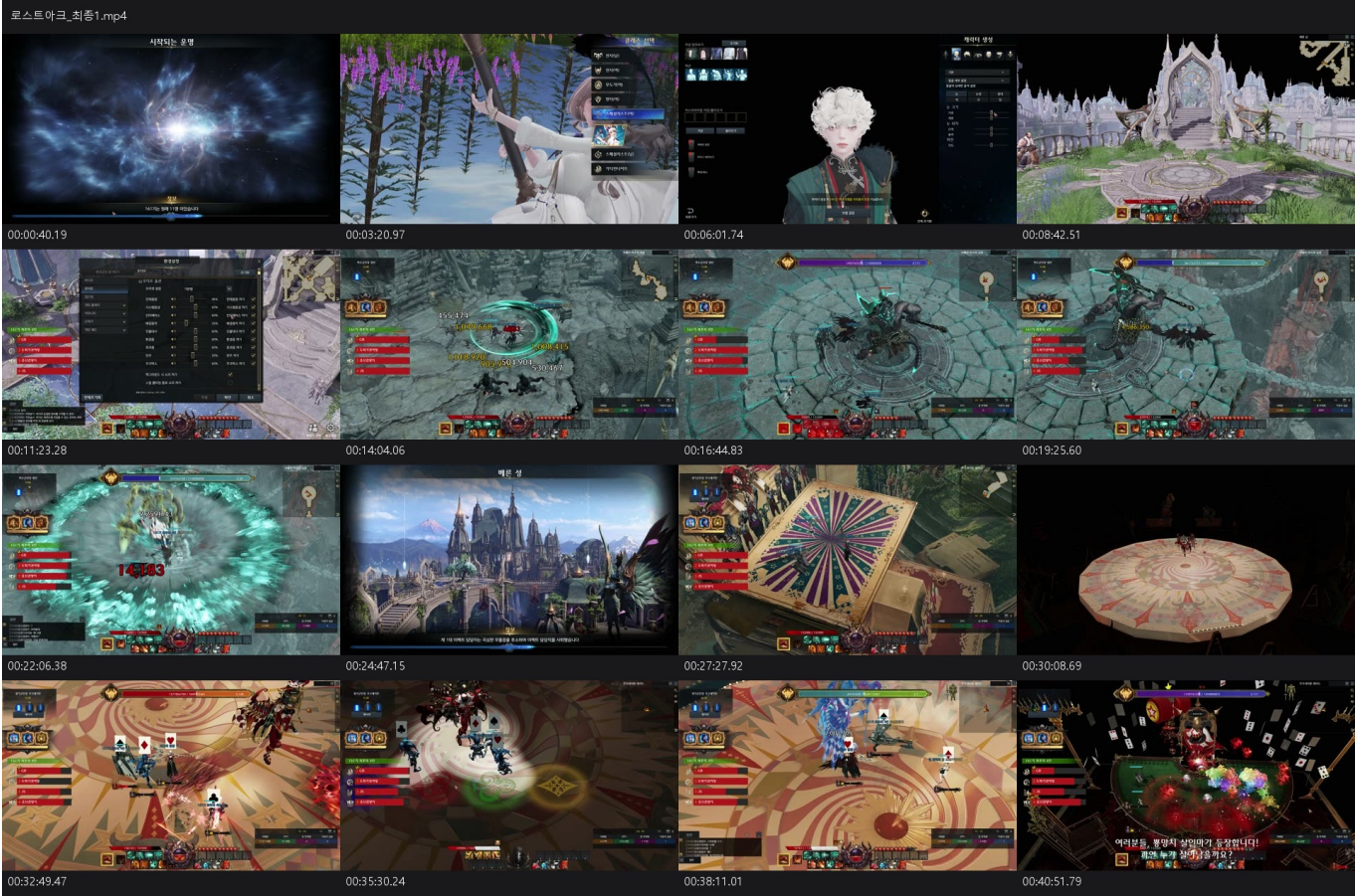
- 이전 대화: '포트폴리오와 엔진 수준 평가', chat ID `01a0fb8a-a58f-7963-9f35-b5cd5c11db4e`. 약 200페이지의 사례 연구형 기술서와 영상 근거의 연결 의도를 계승했다.
- 기존 Notion: <https://app.notion.com/p/3a4b8c3c75e2814b851edb4ef62cf5b0> . 2026-07-25 자료의 구성만 참고했으며 개인 연락처를 본 원고에 재수록하지 않았다.
- 선배 영상 기록: `.md/GB/08-01/2026-08-01_LOSTARK_REFERENCE_TEAM_PORTFOLIO_VIDEO_ANALYSIS.md` 및 `RAW_TRANSCRIPT.md`. 원본 영상 전체를 새로 시청한 것은 아니다.
- 기존 조사: `LostArkTransfer/Sync-20261002/Recovery-20261002-1315/Portfolio-Inventory-Index.json`과 세 domain inventory. 부록은 기존 59개 항목의 정적 근거를 다시 찾기 위한 색인이다.
- 현재 계약: `AGENTS.md`, `CLAUDE.md`, `.md/TEAM/README.md`, `TEAM_GAMEPLAY_INTERFACE_HANDBOOK.md`, `ANIMATION_TOOL_OWNER_HANDOFF.md`, `AREA_DATA_LAYER_GUIDE.md` 및 해당 `PLAN/RESULT`.
- 저장·게시: `.md/GB/10-02/2026-10-02_SAVE_PUBLISH_RUNTIME_GUIDE.md`.
- 성능 상태: `.md/GB/10-02/2026-10-02_FRAME_PIPELINE_OPTIMIZATION_RESULT.md`, `CLASS_MOVIE_PERFORMANCE_RESULT.md`, `MOVIE_ANIMATION_SAMPLE_REUSE_RESULT.md`.
- Hermite 개념: <https://learn.microsoft.com/en-us/windows/win32/api/directxmath/nf-directxmath-xmvectorhermite> . 위치 두 개와 접선 두 개로 구간을 보간하는 API 정의를 확인했다.
- CSM 개념: <https://learn.microsoft.com/en-us/windows/win32/dxtechcharts/cascaded-shadow-maps> . frustum 분할과 구간별 shadow map이 현재 단일 경로와 다름을 확인했다.
- RT 개념: <https://learn.microsoft.com/en-us/windows/win32/direct3d11/d3d10-graphics-programming-guide-output-merger-stage> . render target과 depth/blend 처리의 역할을 확인했다.
- FXAA 개념: https://developer.download.nvidia.com/assets/gamedev/files/sdk/11/FXAA_WhitePaper.pdf . 일반 원리 참고이며 프로젝트 shader가 whitepaper의 모든 구현을 그대로 채택했다는 주장은 아니다.

증거 원장에는 source 확인, 실행 검증, 영상 확인, 개인 기여를 서로 다른 필드로 유지한다. 문서·함수·검사 소스가 존재한다는 것만으로 실행 PASS를 기록하지 않는다.

영상 표본 / 로스트아크_최종1.mp4

원본 길이 00:42:52.367 | 균등 표본 16개 | 인코딩 3840×2160 / 60fps

표기된 시각은 원본 표본의 위치입니다. 컷의 시작·끝, 게임 FPS, 특정 알고리즘의 실행 성공을 증명하는 타임코드가 아닙니다.

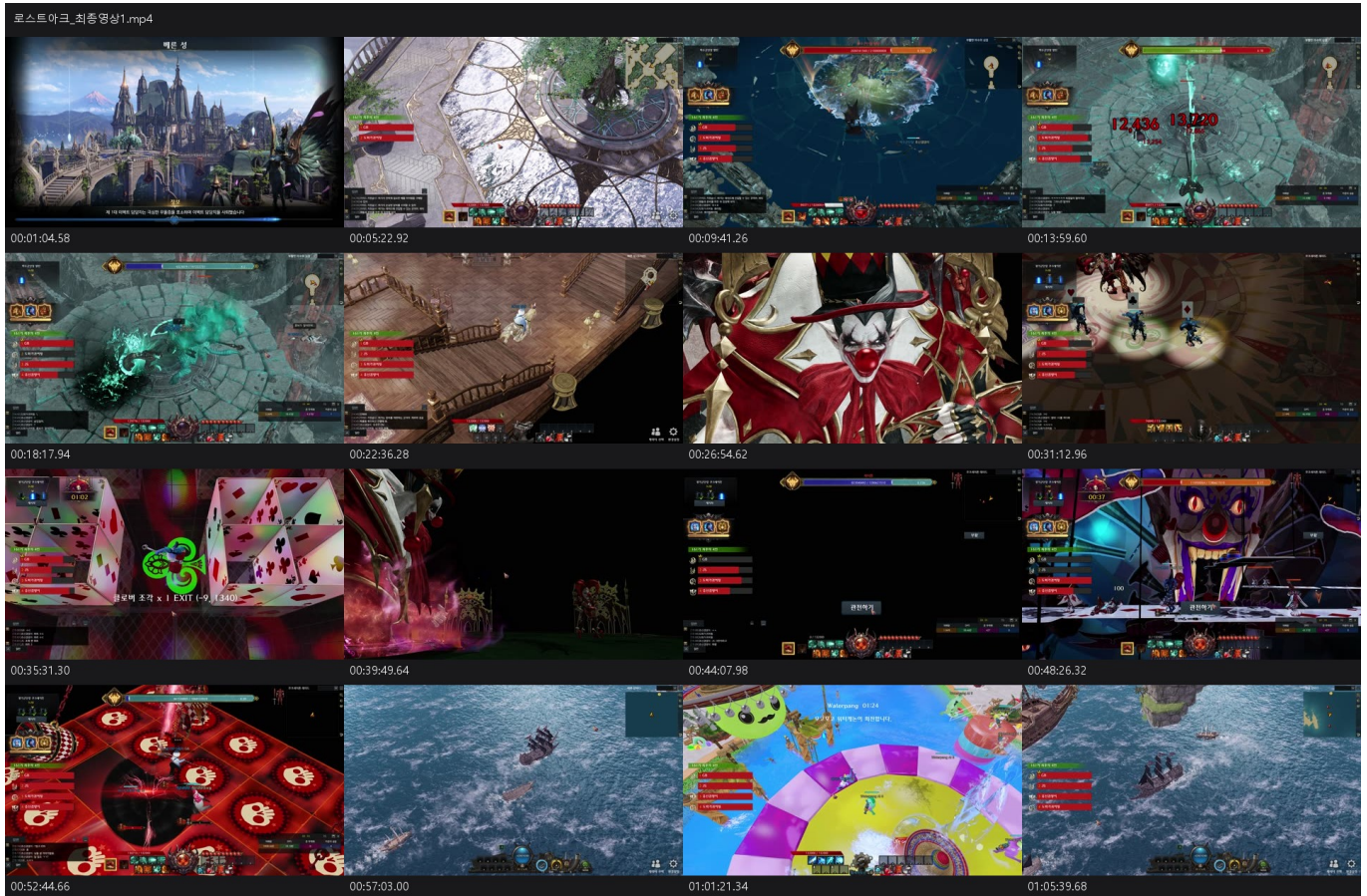


- 00:00:40.19 게임 로딩 화면
- 00:03:20.97 직업 선택과 캐릭터 Movie 장면
- 00:06:01.74 캐릭터 생성 외형 편집
- 00:08:42.51 베른 입장과 HUD
- 00:11:23.28 환경설정 창, 파티 목록
- 00:14:04.06 / 00:16:44.83 / 00:19:25.60 / 00:22:06.38 발탄 레이드 전투
- 00:24:47.15 베른 로딩 화면
- 00:27:27.92 쿠크 진입부
- 00:30:08.69 쿠크 무대 연출
- 00:32:49.47 / 00:35:30.24 / 00:38:11.01 / 00:40:51.79 쿠크 전투와 기믹

영상 표본 / 로스트아크_최종영상1.mp4

원본 길이 01:08:53.433 | 균등 표본 16개 | 인코딩 3840×2160 / 60fps

표기된 시각은 원본 표본의 위치입니다. 컷의 시작·끝, 게임 FPS, 특정 알고리즘의 실행 성공을 증명하는 타임코드가 아닙니다.



00:01:04.58 베른 로딩 화면

00:05:22.92 베른 광장, 파티 목록

00:09:41.26 / 00:13:59.60 / 00:18:17.94 발탄 전투

00:22:36.28 실내 이동

00:26:54.62 쿠키 보스 근접 연출

00:31:12.96 쿠키 전투와 문양 기믹

00:35:31.30 카드미로, 조각 획득 표시

00:39:49.64 쿠키 무대 연출

00:44:07.98 어두운 관전 대기 화면

00:48:26.32 관전 중 힝스크롤 기믹 화면

00:52:44.66 빙고 판과 보스 전투

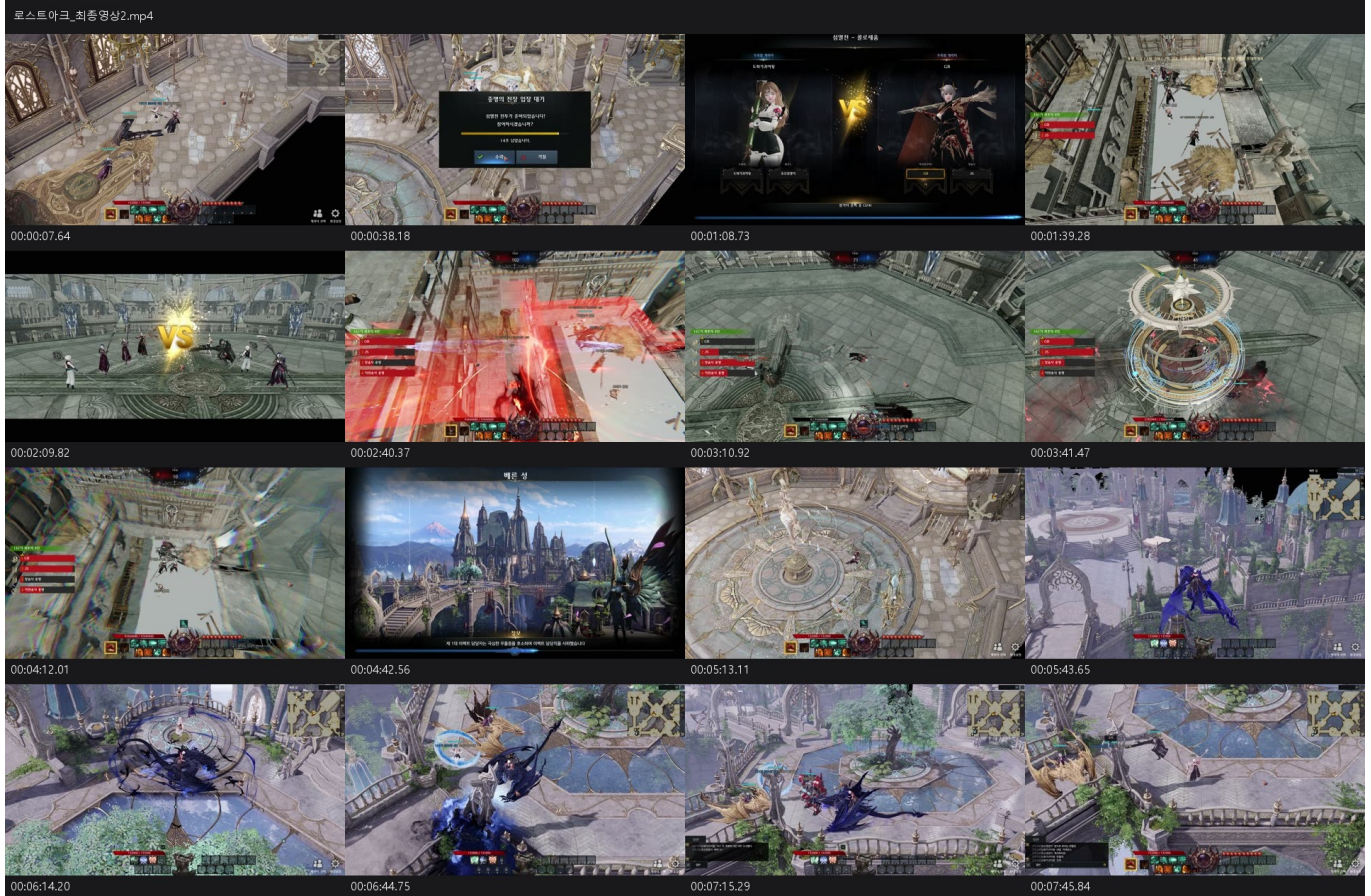
00:57:03.00 / 01:05:39.68 항해

01:01:21.34 마하라카 워터팡 경기

영상 표본 / 로스트아크_최종영상2.mp4

원본 길이 00:08:08.750 | 균등 표본 16개 | 인코딩 3840×2160 / 60fps

표기된 시각은 원본 표본의 위치입니다. 컷의 시작·끝, 게임 FPS, 특정 알고리즘의 실행 성공을 증명하는 타임코드가 아닙니다.



00:00:07.64 베른 실내 이동

00:00:38.18 증명의 전장 입장 수락 UI

00:01:08.73 콜로세움 VS 로딩 화면

00:01:39.28 콜로세움 입장 대기 지역

00:02:09.82 양 팀 도열 VS 연출

00:02:40.37 / 00:03:10.92 / 00:03:41.47 / 00:04:12.01 콜로세움 전투

00:04:42.56 베른 복귀 로딩

00:05:13.11 베른 실내 복귀

00:05:43.65 / 00:06:14.20 / 00:06:44.75 / 00:07:15.29 / 00:07:45.84 베른의 용 탑승·이동·이펙트

부록 / 툴·파이프라인 기능 색인

기존 정적 조사에 기록된 설명과 대표 경로입니다. 현재 화면 성공·개인 기여·모든 코드의 재검증을 뜻하지 않습니다. 오늘 확인한 중요한 정정은 본문과 아래 경계에 반영했습니다.

INV-01 F1 Developer Tools 공통 진입점

F1로 제작 도구 목록을 열고 여러 도구를 함께 표시·숨김·포커스한다.

도구 생성은 EnsureDebugTool에서 지연 수행하며 창 숨김은 domain draft를 버리지 않는다. 입력 소유 도구와 게임 입력 차단을 구분한다.

경계: 공통 F1은 제품 경로에도 존재. Map·Animation·Effect 등 일반 제작 메뉴는 _DEBUG, 두 Boss Tool 메뉴는 Debug/Release 공통.

근거: Client/Private/MainApp.cpp:11387 / Client/Private/MainApp.cpp:14580 / Client/Private/MainApp.cpp:14605

INV-02 Map Tool / Area 편집

Lobby Test 또는 지원하는 현재 Area에서 배경·Deploy·NPC·트리거 등을 편집하고 Save 후 해당 scope를 Publish한다.

catalog asset ID와 placement ID를 분리한다. 저장 전 런타임 표현·Transform을 검증하며, map placements와 world sequence 연결 저장에 lock/recovery를 사용한다. 현재 런타임과 원본 placement 집합이 다르면 편집 연결을 거부한다.

경계: Debug 제작 UI. Save는 Data 저작 정보를 바꾸며 Server 충돌·gameplay나 게시 DataFiles가 자동 변경되는 것은 아니다.

근거: Client/Private/MapTool.cpp:633 / Client/Private/MapTool_Placements.cpp:211 / Client/Private/MapTool_Area.cpp:1008

INV-03 Map Tool Navigation·파괴 확장

Area 내 bake 범위를 지정해 navigation preview/bake를 수행하고 blocker·paint·파괴 상태를 편집·저장한다.

preview와 확정 bake를 분리하고 CNavGridBaker로 배치 geometry를 입력한다. 파괴 draft는 group/binding/pattern 참조를 검증한다. INTACT/FRACTURED/DESPAWNED 상태 preview를 제공한다.

경계: Debug 저작·미리보기. 저장된 입력의 Server 소비는 Navigation/World publisher 경계를 거친다. 로컬 preview 자체가 서버 판정은 아니다.

근거: Client/Private/MapTool_NavigationBake.cpp:174 / Client/Private/MapTool_NavigationBake.cpp:252 / Client/Private/MapTool_Navigation.cpp:642

INV-04 World Level Tool 통합 목록

Area의 Map/Deploy/World Sequence/Composition 항목을 조회하고 원래 편집기로 이동하거나 월드 위치를 포커스한다.

source path가 지정된 Area catalog를 읽어 saved inventory를 구성하고, 현재 owner draft와 구분한다. edit/focus 요청은 typed WORLD_LEVEL_TOOL_REQUEST로 전달한다.

경계: Debug 탐색·편집 연결 도구. 목록 새로고침은 다른 도구 draft나 실제 runtime placements를 강제로 reload하지 않는다.

근거: Client/Private/WorldLevelTool.cpp:171 / Client/Private/WorldLevelTool.cpp:372 / Client/Private/WorldLevelTool.cpp:527

INV-05 Animation Clip Tool / Skill Bindings

Scene Character의 실제 model clip을 재생·검토하고 스킬별 순서 있는 presentation clip을 연결·저장한다.

CPlayerSkillCatalog의 승인된 skillId와 실제 clip 이름으로 문서를 검증한다. 로드 실패 시 현재 문서를 보존하고, 저장은 baseline 비교와 atomic save를 사용한다. combo stage 수는 PlayerSkills 소유다.

경계: Debug 제작 UI. 입력 키·Server damage/timing을 임의로 다시 정의하지 않고 presentation 연결을 저장한다.

근거: Client/Private/Animation_Tool_SkillBindings.cpp:46 / Client/Private/Animation_Tool_SkillBindings.cpp:72 / Client/Private/Animation_Tool_SkillBindings.cpp:242

INV-06 Action Workbench / Sequencer

Boss·Character·Object·Sequence·World 대상과 리소스를 선택하고 timeline의 animation/effect/collider/sound/camera/light 등을 구성한다.

ICompositionWorkbenchSession이 공통 pane UI와 각 domain 문서·draft·저장 소유자를 분리한다.

Begin/End_WorkbenchFrame으로 모든 pane 렌더 뒤 지연 저장을 처리한다. Character/Valtan/Kouku는 독립 세션을 사용한다.

경계: Action Workbench 창은 Debug 제작 UI. 저장된 composition과 제품 재생 경로는 별도이며, Kouku 독립 Sequence workspace는 local preview/Save만 지원한다.

근거: Client/Public/CompositionWorkbenchSession.h:25 / Client/Public/CompositionWorkbenchSession.h:68 / Client/Private/SequencerTool.cpp:1590

INV-07 Action Workbench / Object

오브젝트 motion·transform·collider·effect track을 편집하고 Save와 Publish를 구분한다.

WorldSequence source와 이를 참조하는 Composition occurrence를 연결 저장한다. 저장 전 현재 source를 다시 읽고 편집 field가 외부에서 바뀌면 기존 source/draft를 유지하며 거부한다.

경계: Debug authoring/preview. 별도 Server play controls와 publisher를 통해 제품 반영하며, 로컬 preview가 gameplay 권위는 아니다.

근거: Client/Private/WorldObjectTool.cpp:799 / Client/Private/WorldObjectTool.cpp:945 / Client/Private/WorldObjectTool.cpp:1209

INV-08 Effect Tool V1

resource browser에서 effect를 선택하거나 새 element를 만들고 material/transform/timeline을 편집한 뒤 Save Changes 한다.

현재 문서·Detail draft·product-bound occurrence를 분리한다. source baseline을 비교하는 atomic save와 저장 후 preview/resource 재활성화를 사용한다. immutable imported/runtime view는 Save As로 새 authored ID를 만든다.

경계: Debug 제작 UI. 제품과 연결된 저장은 이후 product spawn에 반영될 수 있지만 이미 실행 중인 occurrence는 기존 immutable resource를 유지한다.

근거: Client/Private/Effect_Tool.cpp:845 / Client/Private/Effect_Tool_Documentlo.cpp:173 / Client/Private/Effect_Tool_Documentlo.cpp:302

INV-09 Effect Tool V2

mesh/DDS resource를 찾아 slot에 연결하고 live preview로 생성·조정한 effect를 effectv2 JSON으로 저장·로드한다.

저장 시 serialize→parse 검증, 문서 ID와 파일 baseline 비교, atomic file write를 수행한다. 로드는 resource binding과 preview 생성 성공 후 상태를 commit하며 실패 시 이전 binding/type으로 복구한다.

경계: Debug 제작 UI와 별도의 V2 runtime document 계약. V1와 동일 구현이라고 묵지 않으며 화면 품질은 미검증이다.

근거: Client/Private/Effect_Tool_V2.cpp:413 / Client/Private/Effect_Tool_V2.cpp:1055 / Client/Private/Effect_Tool_V2.cpp:1467

INV-10 Camera Tool

Valtan/Kouku cue를 선택해 camera keyframe-look-at-시간을 편집하고 preview 후 저장한다.

카메라 cue draft를 먼저 검증한다. Valtan 저장은 전용 save lock, baseline byte 비교, sibling temporary file을 사용해 외부 변경과 충돌을 거부한다. Kouku는 Area Camera source 저장 경로로 위임한다.

경계: Debug 제작 UI. 카메라 presentation 저작이며 Server gameplay timing/authority와 구분한다.

근거: Client/Private/CameraTool.cpp:565 / Client/Private/CameraTool.cpp:1575 / Client/Private/CameraTool.cpp:1674

INV-11 HUD Layout Tool

HUD·Lobby·Class Select·로딩 화면 등의 document를 선택하고 palette에서 slot을 배치해 크기·순서·상태별 이미지를 편집한다.

slot/rect/asset ID 등을 JSON으로 저장한다. 임시 파일을 write-through 교체하고, 로드는 모든 slot을 staged collection에서 검증한 뒤 commit한다. 제품 CHUDRuntimeView가 Data 문서와 Resources 상대 texture를 별도 소비한다.

경계: 편집 UI와 ImGui preview는 Debug 전용. 저장 layout은 제품 runtime view가 사용하며 ImGui 화면 자체를 제품 UI로 사용하지 않는다.

근거: Client/Private/HUDLayoutTool.cpp:39 / Client/Private/HUDLayoutTool.cpp:1035 / Client/Private/HUDLayoutTool.cpp:1895

INV-12 Equipment Authoring Tool

class와 equipment slot/visual set을 선택해 캐릭터 preview에 적용하고 preset을 저장한다.

presentation catalog와 loadout을 stage·검증한 뒤 적용한다. preview class 전환 시 미저장 draft를 보존한다. preset JSON은 authoringOnly=true로 저장된다.

경계: Debug preview·저작 도구. Data/Actors/EquipmentLoadoutPresets.json은 authoring preset이며 서버 장비 수치나 실제 아이템 지급 기능으로 소개하면 안 된다.

근거: Client/Private/EquipmentAuthoringTool.cpp:162 / Client/Private/EquipmentAuthoringTool.cpp:466 / Client/Private/EquipmentAuthoringTool.cpp:700

INV-13 DimensionMaster Guide AI Tool

안내 대사·트리거·collider·콤보/도움 명령·combat 항목을 편집하고 Save와 Publish를 분리한다.

stable prompt/trigger ID와 참조 목록을 사용한다. document wrapper가 baseline/draft를 publisher에 넘겨 비동기 검증·저장을 수행하며 saved source가 바뀌면 Publish를 거부한다. published revision과 Server revision을 나란히 표시한다.

경계: Debug 제작 UI. 코드로 확인한 것은 안내 데이터와 게임 AI 설정 편집이며 생성형 AI 모델 사용을 뜻하지 않는다. 게시 후 실행 Server의 reload/restart는 별도다.

근거: Client/Private/GuideAITool.cpp:119 / Client/Private/GuideAITool.cpp:165 / Client/Private/GuideAIDocument.cpp:73

INV-14 Waterpang AI Tool

현재 Maharaka room의 bot 수·decision/move/skill 주기·공격성을 조회해 Apply 또는 Save + Apply한다.

IPlayerCommandSink를 통한 typed C2S/S2C 요청에 expected revision과 request sequence를 넣는다. revision 충돌·응답 timeout에 draft를 보존하고, 변경 field만 유지하는 rebase를 제공한다.

경계: Debug 조작 UI이지만 적용 권위와 저장은 Server에 있다. Apply는 현재 room, Save + Apply는 Server source 저장까지 수행한다.

근거: Client/Private/MaharakaAITool.cpp:79 / Client/Private/MaharakaAITool.cpp:106 / Client/Private/MaharakaAITool.cpp:140

INV-15 Shared Balance Test / Valtan Authoring

활성 Server 수치를 읽어 field를 수정하고 Save + Apply하거나 Valtan pattern draft를 저장·게시한다.

숫자 수정은 BaseNumericRevision과 before/value를 포함한 제한된 typed patch이며 Server가 검증·저장·게시·활성화한다. Valtan 문서 저장은 durable receipt를 가진 비동기 transaction과 정확한 source revision에 묶인 publisher 재시도를 사용한다.

경계: Balance 실행 코드와 Boss 내부 경로는 Release에도 존재한다. 숫자 live patch와 전체 Valtan canonical publish는 서로 다른 처리다. 단순 Client 파일 reload로 Server 적용 완료 처리하지 않는다.

근거: Client/Private/BalanceTestPanel.cpp:129 / Client/Private/BalanceTestPanel.cpp:153 / Server/Private/ServerBalanceNumericStore.cpp:793

INV-16 Valtan / KoukuSaydon Boss Tool

pattern-flow-bundle을 선택하고 saved/published 데이터로 Complete Play하거나 상태를 확인한다.

Valtan은 현재 Server revision-sound receipt-world generation을 고정한 준비 단계를 거친 뒤 audition을 제출한다.

Kouku는 composition의 Pattern Flow를 atomic save하고 saved pattern publish 요청과 재생을 분리한다.

경계: 두 Boss Tool 진입점은 현재 Debug/Release 공통. 편집·제품 재생·Server 승인 경로가 함께 있으므로 전부 Debug-only라고 쓰지 않는다. 실제 재생 성공은 이번 조사에서 미확인이다.

근거: Client/Private/MainApp.cpp:14605 / Client/Private/ValtanBossTool.cpp:618 / Client/Private/ValtanBossTool.cpp:1133

INV-17 Rendering Workbench

render profile과 material/lighting 기여를 비교하고 고정 조건의 benchmark 결과를 JSON으로 저장한다.

현재 render 설정을 기록해 비교 세션과 복원을 관리한다. SSAO/Bloom/exposure/toner/IBL-material 기여를 구분하며 capture 중 profile 변경을 제한한다.

경계: Debug 진단 UI. 일부 비교는 session-only로 Save/Publish하지 않는다. benchmark 구현 존재가 원본 게임과 시각적 동일함을 증명하지 않는다.

근거: Client/Private/RenderingBenchmark.cpp:144 / Client/Private/RenderingBenchmark.cpp:215 / Client/Private/RenderingBenchmark.cpp:406

INV-18 Composition Profiler

capture를 시작하고 CPU scope-GPU pass-long operation-counter를 확인한 뒤 JSON을 저장한다.

Engine profiler snapshot을 비동기 exporter로 전달한다. CPU inclusive/self와 GPU timestamp interval, incomplete/omitted capture를 구분한다. GPU interval을 utilization으로 해석하지 않도록 표시한다.

경계: Debug의 F1/F7로 창을 열고 Capture를 별도로 시작한다. 이번 조사에서는 새 성능 측정을 실행하지 않았다.

근거: Client/Private/ProfilerTool.cpp:455 / Client/Private/ProfilerTool.cpp:495 / Client/Private/ProfilerTool.cpp:562

INV-19 표준 Product 빌드

Debug/Release를 선택해 표준 runner를 실행하고 빌드 결과와 runtime input 상태를 확인한다.

Engine→Shared→Server→Client 순서를 보장하고 SDK-shader-runtime DLL을 배포한다. missingRuntimeInputs/invalidRuntimeInputs를 빌드 결과와 구분해 기록한다.

경계: 오프라인 컴파일-배포. 기본 Product 경로는 Client/게임이나 publisher/광역 diagnostics를 실행하지 않는다.

근거: Tools/Build/Invoke-BuildAndRegression.ps1:3 / Tools/Build/Invoke-BuildAndRegression.ps1:636 / Client/Default/Client.vcxproj:1428

INV-20 Core / FullDiagnostic 계약 검증

진단 profile을 명시해 관련 source 계약 검사와 C++ harness-Server contract test를 실행한다.

Product와 분리된 경로에서 Effect/Map/Network 검사, harness 프로젝트, Server --contract-test, Party2/4-PhysX-WModel 검증을 선택한다.

경계: 오프라인 검증-진단 실행 도구. 이번 inventory에서는 실행하지 않았고 Product 성공을 전체 진단 PASS로 간주하지 않는다.

근거: Tools/Build/Invoke-BuildAndRegression.ps1:691 / Tools/Build/Invoke-BuildAndRegression.ps1:754 / Tools/Build/Invoke-BuildAndRegression.ps1:873

INV-21 도메인별 증분 게시 orchestration

Client/Server/KoukuSaydon 등 owner를 선택하고 관련 runtime data 작업만 게시한다.

입력·도구·Resources·출력 fingerprint와 receipt를 비교해 유효 작업을 재사용한다. domain lock 뒤 다시 검사하며 실행 도중 source가 바뀌면 성공 receipt를 남기지 않는다.

경계: 오프라인 publisher orchestration. 일반 Product build와 별도로 실제 저작 변경 또는 게시본 문제에 맞춰 사용한다.

근거: Tools/Build/Invoke-BuildDomainOwner.ps1:52 / Tools/Build/BuildDomainPipeline.psm1:540 / Tools/Build/BuildDomainPipeline.psm1:601

INV-22 Gameplay Balance publisher

Player/Skill/Damage/Boss JSON을 편집하고 Validate/Publish한다.

provenance와 Shared의 bootstrap 행·바이트 제한을 검사한다. 임시 bootstrap을 작성하고 mutex/hash 비교/백업 교체로 Server Gameplay.bootstrap에 반영한다.

경계: 오프라인 데이터 게시. Server는 게시 bootstrap을 소비하며 source JSON 편집 자체가 실행 중 Server 적용은 아니다.

근거: Tools/GameplayPipeline/Publish-GameplayBalance.ps1:5122 / Tools/GameplayPipeline/Publish-GameplayBalance.ps1:5833 / Tools/GameplayPipeline/Publish-GameplayBalance.ps1:5890

INV-23 World Gameplay publisher

Area의 Gameplay/SpawnGroups를 편집하고 대상 World 또는 ALL을 검증·게시한다.

stable ID-schema-참조를 검사하고 Server world/spawngroups/encounterprops와 Client NPC presentation/stage marker를 함께 준비한다. source freshness를 재검사하고 실패 시 역순 복구한다.

경계: 오프라인 게시. 게임 entity-spawn-trigger 권위는 Server runtime이며 Client용 출력은 presentation 입력이다.

근거: Tools/WorldPipeline/Publish-WorldGameplay.ps1:564 / Tools/WorldPipeline/Publish-WorldGameplay.ps1:740 / Tools/WorldPipeline/Publish-WorldGameplay.ps1:1731

INV-24 Server Navigation publisher

MapTool navsource/navpaint 또는 uniform JSON을 준비하고 Area별 검증·게시한다.

격자-layer-region-placement 조건을 검증한 뒤 binary grid와 sidecar를 생성한다. Server/Client의 navgrid/navblockers/navpolicy를 Area 단위 stage/교체/복구하며 Server navsurface도 생성한다.

경계: 오프라인 변환·게시. Client 이동 결과를 서버 정답으로 보내는 구조가 아니라 Server용 navigation runtime 입력을 만든다.

근거: Tools/NavigationPipeline/Publish-ServerNavigation.ps1:243 / Tools/NavigationPipeline/Publish-ServerNavigation.ps1:399 / Tools/NavigationPipeline/Publish-ServerNavigation.ps1:1400

INV-25 Map Authoring publisher

MapTool Save 뒤 Area와 Placements/Lights/Effects/CameraShots 등의 scope를 골라 Validate/Check/Publish한다.

Authoring과 Imported catalog를 결합해 Client DataFiles/Map을 생성한다. Check는 정규화된 예상 bytes와 게시본을 비교하며 Publish는 파일 집합 stage/승격/rollback을 수행한다.

경계: 오프라인 게시. 일부 Valtan pickup은 Server World 출력과 묶이며 모든 map 입력을 무조건 다시 게시하는 도구가 아니다.

근거: Tools/MapPipeline/Publish-MapAuthoring.ps1:12 / Tools/MapPipeline/Publish-MapAuthoring.ps1:3918 /
Tools/MapPipeline/Publish-MapAuthoring.ps1:4022

INV-26 UE3 placement 추출 / MapTool scene 생성

원본 package/UModel 경로로 placement를 추출하고 runtime asset manifest와 연결해 catalog:placement-material 문서를 만든다.

schema3 가시성, cm→m 축 변환, quaternion 왕복 검사와 원본 placement ID 기반 64-bit stable ID를 사용한다. 입력 덮어쓰기를 거부하고 scene 출력 묶음을 stage/rollback한다.

경계: 오프라인 원본 데이터 변환. 원본 게임의 모든 렌더 효과 복원·화면 동일성을 뜻하지 않는다.

근거: Tools/LevelPlacementExtractor/extract_ue3_placements.py:1660 / Tools/LevelPlacementExtractor/build_maptool_scene.py:107 /
Tools/LevelPlacementExtractor/build_maptool_scene.py:197

INV-27 모델·재질 변환 및 설치 보조 도구

material variant와 texture dependency를 수집하고 converter/WModel 후처리를 거쳐 Area에 설치한다.

원본 material parameter와 실제 texture 연결을 기록한다. glTF/기존 WModel geometry-handedness·UV/색·bounds를 대조하며 receipt/fingerprint freshness를 재확인한 뒤 디렉토리를 교체한다.

경계: Assimp 기반 import와 WModel 확장을 구분한다. 자산 변환 성공은 원본 재질·화면의 완전한 재현을 증명하지 않는다.

근거: Tools/LevelPlacementExtractor/build_map_material_variants.py:1773 / Tools/LevelPlacementExtractor/build_map_material_variants.py:1854 /
Tools/ModelAssetConverter/cook_wmodel_geometry_contract.py:2165

INV-28 Effect source / V2 binding validators

Effect/Group/Binding JSON과 실제 Resources를 검사하고 누락·부정합을 수정한다.

V1 catalog↔authored ID/version/resource closure를 검사한다. V2는 중복 JSON key·NaN·루트 탈출 경로와 stage/clip/effect 참조를 거부하며 binding pipeline의 resource read-set freshness도 확인한다.

경계: 오프라인 데이터 계약 검증. 정적 검사 통과는 실제 프레임의 시각적 품질 판정과 별개다.

근거: Tools/EffectPipeline/validate_effect_sources.py:1277 / Tools/EffectPipeline/validate_effect_sources.py:1448 /
Tools/EffectToolV2/validate_effect_v2.py:45

INV-29 캐릭터 커스터마이징 UI 생성기

원본 atlas를 준비한 뒤 Python으로 crop 이미지와 UI layout JSON을 생성한다.

DDS 영역을 PNG로 잘라 Resources 상대 asset ID를 반환한다. 원본 1920×1080 rect를 1280×720로 변환하고 stable slot ID·rect·hover layer를 저장한다.

경계: 오프라인 목적별 생성기. D: 원본경로가 하드코딩되고 직접 저장하므로 범용 transactional UI publisher라고 소개하지 않는다.

근거: Tools/CharacterCustomizing/build_customizing_ui.py:18 / Tools/CharacterCustomizing/build_customizing_ui.py:48 /
Tools/CharacterCustomizing/build_customizing_ui.py:321

부록 / Engine·Client 기능 색인

기존 정적 조사에 기록된 설명과 대표 경로입니다. 현재 화면 성공·개인 기여·모든 코드의 재검증을 뜻하지 않습니다. 오늘 확인한 중요한 정정은 본문과 아래 경계에 반영했습니다.

INV-30 Prototype/Clone와 Component 조립

레벨별 원형 객체를 등록하고 Clone으로 GameObject와 Component 인스턴스를 만든다.

unique_ptr 원형 저장소, shared_ptr 인스턴스, GameObject 컴포넌트 맵을 사용한다. 여러 원형을 일괄 등록할 때 중복 키를 먼저 검사하고 staging map의 노드를 merge한다.

경계: Prototype 기반 조립 구조이며 ECS라고 단정하지 않는다. 전체 객체의 수명 누수·동시성 검사는 수행하지 않았다.

근거: Engine/Private/Prototype_Manager.cpp:36 / Engine/Private/Prototype_Manager.cpp:68 / Engine/Public/GameObject.h:62

INV-31 엔진 프레임 수명과 레이어

입력·사운드·오브젝트·물리·레벨·렌더 제출을 정해진 프레임 순서로 처리한다.

CGameInstance facade가 Object/Level/Physics/Renderer 서비스를 연결한다. Priority_Update, Update, Post_Physics_Update, Level Update, Late_Update를 분리하여 프레임 내 오브젝트 갱신·물리 처리·레벨 동작·렌더 제출 순서를 관리한다.

경계: CPU 호출 순서를 확인한 것이며 현재 실행 FPS나 모든 업데이트 비용은 측정하지 않았다.

근거: Engine/Private/GameInstance.cpp:192 / Engine/Private/Layer.cpp:137

INV-32 레벨 registry와 전환 요청

Lobby에서 시작하고 registry의 factory·맵 범위를 이용해 제품 레벨을 준비한다.

CLevelTransitionService에 요청을 보관하고 CMainApp::Apply_LevelRequest가 프레임 경계에서 소비한다. Loading Level은 activation 요청을 제출한다. 실제 registry에는 Lobby, Character Select, Bern, Valtan, Kouku, Development, Maharaka, Colosseum descriptor가 존재한다.

경계: 오래된 문서의 레벨 목록 대신 현재 registry를 기준으로 기록했다. 실제 모든 레벨 진입은 재검증하지 않았다.

근거: Client/Private/MainApp.cpp:1420 / Client/Private/MainApp.cpp:10947 / Client/Private/Level_Loading.cpp:426

INV-33 비동기 자산 로더와 종료 처리

맵·캐릭터·이펙트 준비를 worker에서 수행하고 준비된 자산을 제품 런타임에 전달한다.

Loader와 Effect producer worker, 준비 batch, prototype staging을 사용한다. Free에서 협력 취소 후 5초 join, 동기 I/O 취소 후 추가 5초를 기다리고 초과 시 프로세스 timeout fail-fast를 수행한다.

경계: 종료 경로의 bounded join을 확인했다. 정상 worker 내부에도 별도 대기 경로가 있으므로 모든 대기가 bounded라고 일반화하지 않는다.

근거: Client/Private/Loader.cpp:288 / Client/Private/Loader.cpp:1347 / Client/Private/Loader.cpp:1866

INV-34 맵 catalog-placement-static batch

맵 정의와 배치 인스턴스를 구분하여 필요한 범위만 생성하고 정적 객체를 batch로 제출한다.

MAP_LOAD_SCOPE를 Loader와 runtime이 공유한다. CMapPlacementRuntime은 catalog와 placement를 stage하고 생성 실패 시 임시 객체를 제거한다. 성공 뒤 기존 상태를 교체하며 stable placement ID와 CPU ray picking을 제공한다.

경계: 로딩 transaction과 stable ID 소비를 확인했다. 배치별 culling 정확도·draw call 절감률·화면은 측정하지 않았다.

근거: Client/Private/MapPlacementRuntime.cpp:76 / Client/Private/MapPlacementRuntime.cpp:123 / Client/Private/MapPlacementRuntime.cpp:160

INV-35 통합 CModel과 CMaterial 자산 경로

정적·애니메이션 모델과 재질을 같은 CModel/CMaterial 경로로 준비한다.

CModel은 .wmodel을 binary loader로 연결하며 MODEL_ASSET_LOAD_DESC overload를 제공한다. CMaterial은 typed material data, DDS/TGA/WIC 텍스처와 SourceCharacter 표면 입력을 소비한다. device/path 기반 공유 텍스처 캐시로 같은 키의 중복 로딩을 조정한다.

경계: CModel에는 Assimp 입력 경로도 남아 있다. 퇴역한 CCookedModel/CBinaryAssetObject를 현재 기능으로 쓰지 않는다. 텍스처 캐시의 실제 메모리 절감량은 미측정이다.

근거: Engine/Private/Model.cpp:1028 / Engine/Private/Model.cpp:2952 / Engine/Private/Material.cpp:471

INV-36 골격 애니메이션과 pose sampling

클립 재생·전환 blending·본 계층 갱신 및 특정 시점의 본 pose 조회를 지원한다.

CModel Clone 시 bone/animation instance를 분리하고 geometry/material을 재사용한다. 클립별 bone matrix sampling, 전환 pose 보관, root-motion 표시 보정이 모델 구현에 연결되어 있다.

경계: 클라이언트의 pose 표현 기능이다. Server 이동·damage 판정을 모델 애니메이션이 소유한다고 설명하지 않는다.

근거: Engine/Private/Model.cpp:152 / Engine/Private/Model.cpp:209 / Engine/Private/Model.cpp:315

INV-37 스킬 presentation 바인딩과 서버 action

class skillId에 실제 모델 클립과 단계별 표현을 연결하고 네트워크 action을 캐릭터 표현에 반영한다.

AnimationSkillBindingDocument가 skill catalog와 실제 clip을 검증하며 COMBO 단계 수를 Server-owned 정의와 일치시킨다. CCharacter::Apply_NetworkAction과 presentation timeline이 재생을 연결한다. 저장은 임시 파일 검증 뒤 원자 교체하며 baseline 검사를 지원한다.

경계: 바인딩 저장·검증 구조를 확인했다. 모든 class-slot의 최종 모션 품질이나 저작 coverage 완료는 주장하지 않는다.

근거: Client/Private/AnimationSkillBindingDocument.cpp:1150 / Client/Private/AnimationSkillBindingDocument.cpp:1228 / Client/Private/AnimationSkillBindingDocument.cpp:1317

INV-38 Deferred 렌더링과 후처리

그림자·불투명 표면·광원·투명 이펙트·후처리·UI를 단계별로 합성한다.

D3D11 render target/MRT를 구성하고 Draw에서 Shadow, NonBlend, optional SSAO, Lights, HDR scene, Blend, ScreenPosts, optional Bloom, Final, overlay/UI 순서를 관리한다. viewport resize는 임시 리소스를 stage한 뒤 swap한다.

경계: 해당 옵션을 구현했다는 의미이며 현재 모든 맵에서 켜져 있다는 의미가 아니다. 팀장 저장 옵션·화면 fidelity·GPU 성능 수치는 별도 확인이 필요하다.

근거: Engine/Private/Renderer.cpp:846 / Engine/Private/Renderer.cpp:418 / Engine/Private/Renderer.cpp:916

INV-39 컴파일된 Shader와 재질군 ABI

제품 실행 시 CSO를 읽고 pass/input signature와 프로그램 variant를 검증한다.

CShader가 compiled effect를 D3DX11CreateEffectFromMemory로 연다. 같은 input signature의 input layout을 재사용하고 ProgramVariantPass annotation을 검사한다. Effect_ShaderFamily 실행 표가 mesh/particle carrier와 native ID 구간 별 셰이더를 선택한다.

경계: 빌드-time shader 분할과 runtime ABI 검사이며 화면 중 HLSL 자동 컴파일 기능으로 표현하지 않는다. 개별 native 재질의 원작 동등성은 미검증이다.

근거: Engine/Private/Shader.cpp:250 / Engine/Private/Shader.cpp:355 / Engine/Private/Shader.cpp:371

INV-40 문서 기반 Effect runtime

Mesh/Sprite/Particle/Decal/Trail/Light/ScreenPost 표현을 저작 문서와 실행 carrier로 연결한다.

Product catalog는 stable asset ID로 direct-authored 문서를 해석한다. Prepared renderer는 catalog revision, immutable document identity, material registry generation 또는 resource signature를 검사하고 리소스 staging 성공 뒤 state를 교체한다.

경계: carrier별 지원 범위와 저작 완료 상태는 서로 다르다. 모든 Effect가 복원 완료되었다고 설명하지 않는다.

근거: Client/Public/Effect_AuthoringDocument.h:27 / Client/Private/Effect_DocumentRenderer_Staging.cpp:87 / Client/Private/Effect_DocumentRenderer_Staging.cpp:125

INV-41 Effect prewarm과 지연 spawn

요청된 Product Effect를 준비 큐에 등록하고 준비된 occurrence를 프레임 흐름에 맞춰 생성한다.

CEffectPresentationService가 preparation epoch-catalog revision·실패 revision과 prewarm queue를 유지한다. CMainApp은 Commit_PendingSpawns를 호출하며 Spawn API는 world root-level placement 등 명시적인 presentation 입력을 받는다.

경계: 준비 수명과 commit 경계를 확인했다. 실제 첫 등장 stall 개선량은 이번 조사에서 측정하지 않았다.

근거: Client/Private/Effect_PresentationService.cpp:577 / Client/Private/Effect_PresentationService.cpp:2990 / Client/Private/Effect_PresentationService.cpp:5821

INV-42 제품 UI layout·sprite·pointer 중재

JSON slot 배치·이미지·keyframe 표현을 제품 UI로 만들고 화면 좌표에서 클릭을 중재한다.

CUILayoutRuntime과 CUI_Sprite가 reference resolution을 사용한다. CUIInputRouter는 cursor를 viewport 기준으로 변환하고 front window-한 press 한 action 규칙을 적용하며 UI가 소비한 마우스를 gameplay에서 차단한다. Renderer UI는 layer별 stable_sort로 순서를 유지한다.

경계: UUILayoutRuntime::Load는 파일 없음/JSON parse 실패 시 S_OK로 반환하는 경로가 있다. 모든 UI 로드가 strict transaction-실패보고를 완전히 지킨다고 일반화하지 않는다.

근거: Client/Private/UUILayoutRuntime.cpp:106 / Client/Private/UUIInputRouter.cpp:37 / Client/Private/UUIInputRouter.cpp:98

INV-43 입력 intent와 typed command 경계

이동·스킬·상호작용 입력을 서버 명령으로 제출하고 UI/카메라 상태에 따라 차단한다.

CPlayerController는 IPlayerCommandSink에 Request_MoveGoal/Request_UseSkill 등 typed intent를 보낸다. class와 quick-slot은 CPlayerSkillCatalog::Find_BySlot으로 해석하며 UI mouse claim을 검사한다. 지면 지정 스킬은 별도 typed 호출로 확인 입력을 구분한다.

경계: command 전달 구조를 확인했다. 실제 네트워크 지연·스킬 승인 성공률이나 모든 입력 예외 경로는 재검증하지 않았다.

근거: Client/Public/PlayerCommandSink.h:14 / Client/Public/PlayerCommandSink.h:110 / Client/Private/PlayerController.cpp:810

INV-44 Replication presentation과 전투 HUD

수신 world/player 상태를 캐릭터·boss·인벤토리·HUD 표현에 반영한다.

CClientReplication이 네트워크 event를 소비하며 world snapshot과 spawn/despawn을 적용한다. CCombatHUDViewModel은 player/boss snapshot을 받아 read-only Get_Player/Get_Boss view를 제공해 화면과 packet 처리를 분리한다.

경계: 클라이언트 표현 계층이다. damage-world 이동의 권위를 클라이언트에 귀속시키지 않는다. 멀티 PC 실행 검증은 별도다.

근거: Client/Private/ClientReplication.cpp:352 / Client/Private/ClientReplication.cpp:448 / Client/Public/CombatHUDViewModel.h:533

INV-45 Animation Tool의 실제 target 연결

씬 캐릭터와 preview 모델을 구분하여 도구가 실제 본·모델·pose를 참조한다.

CAnimationTargetService는 weak_ptr target과 generation을 관리한다. target 교체 시 generation을 변경하고 historical pose binding에서 예상 generation을 검사하여 오래된 모델 참조를 배제한다.

경계: target 수명과 참조 경계를 확인했다. 본 부착 위치와 최종 영상 품질은 실제 모델·사용자 화면 확인이 필요하다.

근거: Client/Private/AnimationTargetService.cpp:10 / Client/Private/AnimationTargetService.cpp:40 / Client/Private/AnimationTargetService.cpp:367

INV-46 Resources root와 자산 전달 경계

runtime asset ID를 Resources 상대경로로 해석하고 루트 밖 경로를 거부한다.

CRuntimeAssetRoot는 환경변수와 실행 파일 주변 root를 해석하고 ResolveInsideRoot에서 절대·root-qualified-canonical escape를 차단한다. Git 소스/DataFiles와 외부 runtime Resources, EXE/DLL/CSO 빌드 산출물의 전달 경계를 분리한다.

경계: 경로 해석 정책이다. ZIP manifest나 전체 pack SHA를 runtime 자산의 정본 계약으로 주장하지 않는다.

근거: Client/Private/RuntimeAssetRoot.cpp:83 / Client/Private/RuntimeAssetRoot.cpp:113 / Client/Private/RuntimeAssetRoot.cpp:150

INV-47 CPU/GPU 프로파일링 계측

프레임·작업·모델·렌더 단계별 계측과 capture snapshot을 제공한다.

CProfiler의 CPU scope, GPU query scope, pipeline 계측, capture epoch와 work token을 사용한다. 엔진 update/render에 scope가 실제 연결되어 있어 진단 화면이 읽을 수 있는 기록을 만든다.

경계: 계측 인프라의 존재를 확인했다. 측정 캡처 없이 FPS 향상이나 GPU 최적화 퍼센트를 포트폴리오에 쓰면 안 된다.

근거: Engine/Private/Profiler.cpp:196 / Engine/Private/Profiler.cpp:281 / Engine/Private/Profiler.cpp:471

INV-48 제품 기능과 저작 도구의 통합

게임 런타임 위에서 Map/Animation/Effect/World Object/Rendering Workbench 등 저작 도구를 연결한다.

CMainApp이 도구 Update와 Developer Tools hub를 연결한다. 도구는 제품 캐릭터·맵·이펙트 서비스와 데이터 정본을 소비하며, audition 전용 후보는 Product Effect catalog와 별도 registry-freshness gate로 격리된다.

경계: 도구별 Debug/Release 범위와 audition/product admission 상태는 따로 확인한다. 과거 freshness 결과를 오늘의 전수 검사로 승격하지 않는다.

근거: Client/Private/MainApp.cpp:2604 / Client/Private/MainApp.cpp:2652 / Client/Private/MainApp.cpp:15016

부록 / Server-Shared 기능 색인

기존 정적 조사에 기록된 설명과 대표 경로입니다. 현재 화면 성공·개인 기여·모든 코드의 재검증을 뜻하지 않습니다. 오늘 확인한 중요한 정정은 본문과 아래 경계에 반영했습니다.

INV-49 서버 권위 시뮬레이션과 네트워크 계층 분리

ServerApp이 Winsock/listener/session 수명과 accept/room thread를 소유한다. Room_Loop의 1/30초 고정 간격으로 GameRoom을 갱신하며 이동·전투·월드 상태를 서버에서 확정한다.

네트워크 수신과 게임 규칙을 session → command → GameRoom → replication 경계로 나눈 구조.

경계: 두 Client의 이동·스킬 결과가 같은 Server 상태를 반영하는지 사용자 플레이로 확인. 현재 세션에서 다중 Client 검증은 수행하지 않았다.

근거: Server/Private/ServerApp.cpp:2627 / Server/Private/ServerApp.cpp:2705 / Server/Private/GameRoom.cpp:840

INV-50 TCP 프레이밍과 송신 큐의 과부하 처리

길이와 packet type으로 TCP stream을 frame으로 복원한다. 세션별 송수신 경로와 bounded outbound queue를 사용하고, 밀린 world snapshot은 최신 것으로 합치며 reliable frame 공간을 예약한다.

snapshot freshness와 reliable 메시지 순서를 구분하며 큐 크기·byte-drop-coalescing 지표를 기록한다.

경계: 구현은 Winsock select/nonblocking socket과 session thread 방식이다. IOCP나 대규모 동접 성능 검증으로 설명하지 않는다.

근거: Shared/Public/Network/PacketFrame.h:1 / Shared/Public/Network/PacketStreamParser.h:1 / Server/Private/ClientSession.cpp:520

INV-51 세션별 캐릭터 선택 공간과 공유 월드

CHARACTER_SELECT_ARENA 진입은 해당 session 전용 CGameRoom을 생성한다. 다른 world는 shared simulation을 조회한다. 입장 바인딩은 session 종료 상태 및 gameplay revision을 함께 검사한다.

같은 simulation 구현을 사용하면서 private arena와 shared world의 수명·방송 범위를 구분한다.

경계: 서로 다른 캐릭터 선택 session이 상대 actor나 전투 상태를 보지 않는지 별도 실행 확인이 필요하다.

근거: Server/Private/ServerApp.cpp:3889 / Server/Private/ServerApp.cpp:3943 / Shared/Public/Network/PacketType.h:131

INV-52 4인 파티의 원자적 월드 이동

모든 파티원 입장 후보, navigation/profile 검사, 초기 송신 frame 및 관찰자 알림을 준비한 뒤 기존 membership을 변경한다. 준비 실패는 기존 방·파티를 유지하며 typed transfer failure를 전달한다.

Stage_PlayerEntry와 bounded reliable batch reservation 후 membership swap/commit으로 부분 이동을 막는다.

경계: 파티 레이드 입장과 한 명의 입장 거절 상황을 캡처하면 구조 설명의 근거가 된다. 이 세션에서는 해당 재현을 실행하지 않았다.

근거: Shared/Public/Network/PacketType.h:632 / Server/Private/GameRoom_PartyWorld.cpp:1069 / Server/Private/GameRoom_PartyWorld.cpp:1276

INV-53 데이터 기반 스킬·콤보·쿨다운·피해 판정

PlayerSkillSystem이 진행 중 action, combo window, cooldown, resource, stance 조건을 검사하고 확정된 skill state를 저장한다. ServerCombatHitRuntime이 피해·버프·hit event를 계산한다.

클라이언트의 입력/표현과 서버의 수치 및 전투 판정을 분리한다. Data/Balance의 정의가 Server catalog로 소비된다.

경계: 직업별 skillbinding 애니메이션과 서버 확정 콤보·쿨다운 표시가 함께 동작하는 장면을 사용자 확인 대상으로 둔다.

근거: Server/Private/PlayerSkillSystem.cpp:518 / Server/Private/PlayerSkillSystem.cpp:572 / Server/Private/PlayerSkillSystem.cpp:673

INV-54 서버 navigation 및 경로 탐색

영역별 grid를 선택하고 이동 가능 cell과 높이·traversal 조건을 검사한다. 8방향 A*와 대각선 corner 통과 방지, line-of-sight 기반 경로 보정이 있다. 검색 scratch를 thread_local generation stamp로 재사용한다.

query 결과는 재사용하지 않으면서 allocation을 줄이고 query 시간·확장 노드 지표를 계측한다. 중첩 호출에는 별도 scratch를 사용한다.

경계: 이 경로 탐색을 Detour/navmesh 구현으로 설명하지 않는다. 성능 개선율은 이 세션에서 측정하지 않았다.

근거: Server/Private/ServerNavigation.cpp:846 / Server/Private/ServerNavigation.cpp:1137 / Server/Private/ServerNavigation.cpp:1326

INV-55 레이드 flow와 월드 상태 동기화

GameRoom의 고정 tick 안에서 보스·전투 오브젝트·Kouku player mode/raid flow·월드 파괴 상태를 갱신한다. Kouku flow는 게시된 pattern/catalog를 preflight한 뒤 PREPARING, CINEMATIC, COMBAT 등 phase로 진행한다.

보스 연출과 gameplay 상태를 안정적인 pattern/flow ID와 server tick 기준으로 연결한다.

경계: Valtan·Kouku 관문, 미니게임, 지형 변화와 Client presentation의 동기화는 실제 화면으로 확인해야 한다.

근거: Server/Private/GameRoom.cpp:1232 / Server/Private/GameRoom.cpp:1240 / Server/Private/GameRoom.cpp:1256

INV-56 게임플레이 경로를 재사용하는 AI와 저작 설정

Guide와 Waterpang AI를 별도 control kind의 SERVER_PLAYER로 표현한다. Guide owner 시작/종료, Waterpang bot 생성·match update를 서버가 처리한다. AI tuning GET/APPLY/SAVE에는 expected revision 검사가 있다.

가짜 network session을 만들지 않고 기존 이동·스킬·피격·복제 구조를 소비하며 저작 변경 충돌을 revision으로 감지한다.

경계: 기본 Waterpang botCount 20은 설정값이며 동접 20인 네트워크 부하 테스트 결과가 아니다.

근거: Shared/Public/Network/PacketType.h:207 / Server/Private/GameRoom_Guide.cpp:48 / Server/Private/GameRoom_Guide.cpp:79

INV-57 인벤토리·장비·상점 및 세션 상태 복원

Server가 아이템 지급·사용, 장비 장착·수리, 구매와 캐릭터 상태 복원을 검증하고 적용한다. 월드 이동에서는 inventory/purse를 입장 후보에 포함한다.

UI가 재화나 장비 수치를 직접 확정하지 않고 typed request의 서버 결과를 표시하도록 분리한다.

경계: 캐릭터 roster는 process-session 상태다. 계정 인증, 영구 DB 저장, 상용 경제 시스템을 구현한 것으로 확대하지 않는다.

근거: Server/Private/GameRoom_Inventory.cpp:28 / Server/Private/GameRoom_Inventory.cpp:96 / Server/Private/GameRoom_Inventory.cpp:293

INV-58 콜로세움 matchmaking과 경기 상태

queue join/leave, match 구성, load-ready, return 요청, 경기 phase와 score 집계를 서버가 관리한다. Colosseum recruitment와 AI 상태가 서버 room 경로에 포함된다.

클라이언트 로딩 완료와 경기 시작/종료를 protocol message와 server match state로 연결한다.

경계: 매칭·로딩·경기 종료·Bern 복귀 화면은 사용자 확인 대상으로 남긴다.

근거: Server/Private/GameRoom_Inventory.cpp:990 / Server/Private/GameRoom_Inventory.cpp:1095 / Server/Private/GameRoom_Inventory.cpp:1180

INV-59 게임플레이 데이터 세대 전환

ServerApp은 준비된 수치 balance generation을 각 room에 stage하고 commit하는 경로를 가진다. 입장 시 활성 generation revision도 검사한다.

데이터 reload를 개별 field의 즉시 덮어쓰기 대신 일관된 generation 교체로 다룬다.

경계: 정적 구현 확인이며 이번 작업에서 라이브 reload 성공/실패 주입은 수행하지 않았다.

근거: Server/Private/ServerApp.cpp:5752 / Server/Private/ServerApp.cpp:5788 / Server/Private/ServerApp.cpp:3943